# Book Recommendation based on Large Language Model

**M V Sri Lasya**
**2023003411**

**M. Sai Dheeraj**
**2023002482**

**School of Science**

**GITAM Deemed to University**

**Hyderabad Campus**

**M. Sc. Data Science 2025**

# BOOK RECOMMENDATION SYSYTEM USING LARGE LANGUAGE MODEL

By

M.V. Sri Lasya — 2023003411

M. Sai Dheeraj — 2023002482

**Submitted in Partial fulfilment of the Requirements for the Degree of Master of Science in Data Science at GITAM Deemed to be University**

Under Supervisor

Dr. Motahar Reza

**School of Sciences**
**GITAM Deemed University**
**Hyderabad Campus**

**April 2025**

# Declaration and Approval

## Declaration

We certify that this dissertation/Major Project is my own original work and has not been previously submitted or approved for the award of a degree at this or any other university. To the best of my knowledge, it does not include any material authored or published by others, except where appropriate citations and references are given.



M.V. Sri Lasya
2023003411
M.Sc Data Science
GITAM Deemed University

M. Sai Dheeraj
2023002482
M.Sc Data Science
GITAM Deemed University

## Approval

The dissertation of M.V. Sri Lasya and M. Sai Dheeraj was reviewed and approved for examination by the following:

Dr. Motahar Reza,

Department of Mathematics
School of Sciences,
GITAM University, Hyderabad

Campus External Expert,

Principal,
School of Science,
GITAM University

# Acknowledgements

We would like to express my sincere gratitude to my project supervisor, **Dr. Motahar Reza**, for his invaluable guidance, constant support, and encouragement throughout the course of my major project. His expertise and insights have played a crucial role in shaping this work, and we are truly grateful for his mentorship.

We extend my heartfelt thanks to **Dr. Motahar Reza**, Principal, School of Science, GITAM University, Hyderabad, for providing a stimulating academic environment and unwavering support. We are also deeply grateful to **Dr. Abhijit Patil**, Project Coordinator, for his assistance, constructive feedback, and motivation, which have been instrumental in the successful completion of this project.

We would also like to acknowledge **Prof. D. S. Rao**, Pro Vice-Chancellor, GITAM Hyderabad Campus, for his continuous support and for fostering an environment that encourages academic excellence and research.

We are grateful to my **professors and mentors**, whose teachings and guidance have been fundamental in shaping my knowledge and skills in Data Science. Their support has been invaluable in my academic journey.

Furthermore, we are profoundly thankful to my **parents and family** for their unconditional love, encouragement, and sacrifices, which have been my greatest source of strength. A special note of appreciation goes to my **friends** for their continuous support, insightful discussions, and encouragement, making this journey more enriching and memorable.

Finally, we express our gratitude to everyone who have contributed to our learning experience at the **School of Science, GITAM University, Hyderabad**. Their support has been instrumental in the successful completion of this project.

M.V. Sri Lasya
2023003411
M.Sc Data Science
GITAM Deemed University

M. Sai Dheeraj
2023002482
M.Sc Data Science
GITAM Deemed University

# Abstract

In the era of digital transformation, delivering personalized content has become a cornerstone of modern applications, significantly enhancing user satisfaction and engagement. One such domain that benefits immensely from intelligent personalization is book discovery. This project presents a novel AI-powered Book Recommendation System that leverages a fine-tuned Large Language Model (LLM) to generate interactive, context-aware, and highly personalized book suggestions.

At the core of this system lies a carefully designed multi-phase pipeline. The first phase involves the construction of a rich and diverse dataset by utilizing the Google Books API, which retrieves extensive metadata for thousands of books. This metadata includes titles, authors, categories, descriptions, publication dates, average ratings, and user reviews. This diverse information base forms the backbone for learning user preferences and book similarities.

The second phase focuses on data preprocessing and fine-tuning of a generative LLM. For this project, the Falcon 3-1B decoder-only transformer model was fine-tuned on a specially formatted instruction-based dataset, allowing it to understand and respond to book-related queries with improved semantic awareness. This fine-tuning enables the model to go beyond surface-level recommendations and respond to complex and nuanced inputs in natural language.

To ensure the quality and relevance of recommendations, the system incorporates embedding-based similarity search using FAISS (Facebook AI Similarity Search). Separate vector representations are created for titles, descriptions, and genres using pre-trained sentence transformers. This enables the system to compare user queries against book embeddings and return the most contextually appropriate matches, even when the queries are vague or do not exactly match any specific book title.

The third phase involves deploying the model through a user-friendly interface built with Streamlit, enabling real-time interaction and feedback. The web interface allows users to input free-form queries, such as themes, moods, or preferences, and receive book suggestions that feel tailored and conversational.

This integration of cutting-edge language models with effective retrieval techniques bridges the gap between automation and personalization. By fine-tuning LLMs for specific tasks and combining them with efficient search and ranking mechanisms, the system delivers not just accurate recommendations but also an engaging and intuitive discovery experience for readers.

Ultimately, this project demonstrates how advanced natural language processing and recommendation techniques can be combined to build systems that truly understand and serve users, redefining how individuals explore and connect with literature in the digital age.

# Index Of the Report

## Table of Contents

# List Of Tables

# List Of Figures

# List of abbreviations

LLM: Large Language Model.

SLM: Small Language Model.

GPT: Generative Pre-Trained Transformer.

FAISS: Facebook Artificial Intelligence Similarity Search.

BERT: Bidirectional Encoder Representations from Transformers.

NLP: Natural Language Processing.

TFIDF: Term Frequency-Inverse Document Frequency.

CBOW: Continuous Bag of Words.

API: Application Programming Interface.

SG: Skip Gram.

MLM: Masked Language Model.

NSP: Next Sentence Prediction.

IVF: Inverted File Indexing.

HNSW: hierarchical navigable small world.

# 1 Introduction

In an age where users are constantly inundated with vast amounts of digital content, the ability to deliver personalized and meaningful recommendations has become more critical than ever. The explosion of information, while empowering, also creates a paradox of choice—making it increasingly difficult for users to find content that truly resonates with their interests. Nowhere is this challenge more evident than in the realm of books, where millions of titles compete for readers' attention across genres, formats, and platforms.

Traditional book recommendation systems, often reliant on collaborative filtering or content-based methods, can fall short in capturing the subtle nuances of user intent, context, and evolving preferences. As a result, there is a growing need for more intelligent, conversational, and context-aware systems that not only suggest relevant books but also engage users in a more natural and interactive way.

This project seeks to address this gap by harnessing the power of Large Language Models (LLMs) and modern information retrieval techniques to build an AI-powered Book Recommendation System. By fine-tuning a decoder-only LLM and combining it with semantic search through vector embeddings, the system aims to offer highly personalized, dynamic, and human-like book discovery experiences.

Through a structured pipeline that includes data collection, model fine-tuning, vector-based similarity search, and intuitive user interaction, this system reimagines how readers can connect with books that match their unique tastes, moods, and reading goals. The following sections of this report delve into the architecture, implementation, and innovations behind the system, showcasing how deep learning and natural language understanding can be leveraged to transform the way we explore literature in the digital era.

## 1.1 Background

Book recommendation systems have emerged as essential tools for enhancing user experience in digital platforms by delivering personalized content tailored to individual preferences. The importance of such systems continues to grow with the increasing volume of available literature and the need to navigate this vast landscape efficiently. Traditionally, book recommendations were generated using simple content-based filtering techniques that leveraged structured metadata such as genre, author, and summaries. However, these early systems often produced repetitive and less nuanced suggestions, limiting their effectiveness and adaptability.

Recommendation systems are broadly classified into three categories: content-based filtering, which focuses on the attributes of books to suggest similar items; collaborative filtering, which utilizes user preferences and behaviours to recommend titles favoured by similar readers; and hybrid filtering, which integrates both approaches to improve recommendation quality. The introduction of collaborative filtering marked a significant milestone in personalization.

Despite its success, this method suffers from issues like the cold-start problem, where new users or books lack sufficient interaction history to generate meaningful recommendations.

Recent advances in machine learning have significantly improved the sophistication of recommendation engines. Natural Language Processing (NLP) techniques enable a deeper understanding of book content, descriptions, and user-generated reviews. These developments have contributed to more context-aware and semantically rich recommendation experiences.

A major leap in this domain has come with the advent of Large Language Models (LLMs), such as the GPT and Falcon families. These models, built upon Transformer architectures, can understand natural language queries, interpret user intent, and generate human-like responses. Their capacity to capture the emotional and contextual nuances of user input has redefined the scope of book discovery, making the process more interactive and engaging.

However, we have noticed that the research in the recommendation systems is not making a fully AI enabled recommendation engine and utilizing the decoder part of the transformers to give a conversational recommendation chatbot.

## 1.2 Problem Statement

In the current digital landscape, where an overwhelming number of books are available online, identifying titles that align with individual preferences can be a daunting task. This project aims to address that challenge by developing an intelligent, AI-driven chatbot designed to help users discover books that resonate with their unique interests and reading habits.

The proposed system delivers personalized book recommendations based on user input such as preferred genres, favorite authors, and thematic preferences. By leveraging advanced deep learning techniques, including the capabilities of Large Language Models (LLMs), the chatbot can understand user context and generate accurate, context-aware suggestions. These models, built on Transformer architectures, enable the system to comprehend natural language and interpret intent effectively, making the recommendation process more meaningful and engaging.

The primary objective of this project is to create a smart, user-friendly solution that simplifies the book discovery process. Through an intuitive conversational interface, the system transforms the search for a new read into a seamless and enjoyable experience.

## 1.3 Research Objectives

As part of this project, we have explored the domain of recommendation systems with a particular focus on the application of Large Language Models (LLMs) such as BERT and other transformer-based architectures. Our objective is to design a book recommendation system that leverages the capabilities of LLMs to generate highly personalized and contextually relevant suggestions.

The system aims to enhance book metadata, particularly descriptions, by enriching them through natural language processing techniques such as word embeddings and semantic analysis, thereby providing more meaningful inputs to the model. We are fine-tuning pre-trained language models to better capture individual user preferences, ensuring that recommendations align more closely with each user's reading habits and interests. In parallel, we are experimenting with lighter-weight models and efficient fine-tuning strategies like LoRA and adapter layers to assess their performance and adaptability in low-data scenarios.

Our implementation utilizes transformer-based architectures, known for their effectiveness in identifying and attending to the most relevant aspects of textual data through self-attention mechanisms. This attention mechanism allows the system to make informed connections between user preferences and available book content, resulting in more accurate recommendations.

In addition to development, we have conducted an in-depth review of recent research literature to understand the current capabilities, optimization strategies, and limitations of LLMs in recommendation tasks. This critical analysis informs our approach to improving model performance and recommendation quality.

Ultimately, the goal is to integrate state-of-the-art language modelling techniques into a robust book recommendation system that delivers personalized, accurate, and meaningful suggestions tailored to individual users.

## 1.4 Research Questions

1. How can transformer-based architectures like BERT and GPT be effectively fine-tuned to improve the accuracy of book recommendations?
2. What role does semantic enrichment of book metadata (e.g., descriptions, genres, user reviews) play in enhancing the quality of LLM-driven recommendations?
3. How do LLM-based recommender systems perform in low-data or cold-start scenarios compared to traditional collaborative and content-based filtering methods?
4. Which fine-tuning strategies (e.g., LoRA, adapters, prefix tuning) offer the best trade-off between performance and computational efficiency in book recommendation tasks?
5. How can user preferences expressed in natural language (e.g., "I like thought-provoking dystopian novels") be interpreted and matched to relevant book content using LLMs?
6. What are the comparative advantages of using encoder-only models (like BERT) versus decoder-only or encoder-decoder models (like GPT and T5) for personalized book recommendation?
7. Can generative LLMs enhance user engagement in book discovery by providing conversational recommendations rather than static lists?
8. What ethical issues arise when using LLMs in recommendation systems, particularly regarding bias in recommendations and explainability?
9. To what extent do multimodal inputs (e.g., images, audio summaries, user annotations) enhance the accuracy and user satisfaction of book recommendations?
10. How does the integration of user behavioural data (e.g., ratings, click-through rates, reading history) with LLM-generated suggestions affect the system's personalization performance?

## 1.5 Justification

In the modern digital landscape, where millions of books are readily accessible through online platforms, the task of identifying content that aligns with an individual's reading preferences has become increasingly complex. Traditional recommendation methods, such as collaborative and content-based filtering, often fall short in capturing the nuanced and evolving tastes of users, leading to generic or repetitive suggestions that reduce satisfaction and engagement.

This project is justified by the need to enhance the book discovery experience through intelligent, adaptive systems that can understand user preferences at a deeper, more contextual level. Recent advancements in artificial intelligence, particularly in deep learning and large language models (LLMs), present a timely opportunity to revolutionize recommendation systems by moving beyond surface-level metadata to semantically rich user understanding.
By integrating an AI-powered chatbot, the system transforms book discovery from a static process into an interactive, personalized journey.

The use of LLMs enables the recommendation engine to interpret nuanced natural language inputs and generate suggestions that are emotionally and semantically aligned with the user's intent. This conversational interface design has been shown to increase recommendation relevance and user satisfaction in recent works on LLM-based conversational recommenders

Moreover, LLMs facilitate recommendations even in cold-start or sparse data scenarios by leveraging pretrained semantic understanding to generate item descriptions and infer preferences where traditional models fail. The system's adaptive learning capability further ensures continuous refinement based on user interactions, improving its recommendations dynamically over time

In summary, this project directly addresses the limitations of traditional recommendation systems by leveraging cutting-edge AI technologies. It offers a more effective, efficient, and enjoyable way for users to discover literature that resonates with their unique preferences, reflecting the growing shift in recommender system architecture toward LLM-powered, user-centric designs.

## 1.6 Scope

The focus of this project is the development and deployment of an intelligent Book Recommendation Chatbot, designed to offer personalized book suggestions that cater to individual user preferences. This system utilizes a fine-tuned Large Language Model (LLM) and advanced embedding techniques to generate precise, contextually relevant recommendations, ensuring a highly tailored user experience. Leveraging recent advances in recommendation systems, the chatbot aims to bridge the gap between traditional recommendation methods and modern LLM-based approaches.

The user interface will be accessible through a web-based platform, developed using frameworks such as Streamlit, ensuring a seamless, responsive, and user-friendly experience. The design of the interface is aimed at offering accessibility to a wide range of users, from casual readers to dedicated book enthusiasts. Such frameworks are particularly suited for building interactive recommendation systems, as they allow for real-time interaction with minimal friction. Ensuring ease of use is paramount for the system's success, as intuitive user interfaces have been shown to enhance user satisfaction in similar applications.

At the heart of the recommendation engine is the data processing and embedding system, which will leverage curated metadata extracted from the Google Books API. This metadata includes crucial book information, such as titles, authors, genres, descriptions, and user ratings. The system will process this data to generate embeddings that capture semantic relationships between books, enabling the system to align book recommendations with user preferences. Embedding-based techniques are particularly effective for improving the relevance of recommendations by understanding the underlying semantic structure of the content.

The core functionality of the system revolves around generating personalized recommendations based on user inputs, such as reading preferences, favorite genres, and author interests. Through continuous user interaction, the system will learn and adapt to evolving preferences, enhancing its accuracy over time. This iterative learning process is central to the chatbot's ability to refine its recommendations dynamically, a feature that aligns with the latest developments in machine learning for personalized recommendation systems.

The system's ability to learn from user feedback and interaction patterns will allow it to continuously refine its recommendations. As users engage with the chatbot, the system will improve its understanding of user preferences, thus providing increasingly accurate suggestions. This dynamic adaptation is a critical aspect of modern recommendation systems that utilize LLMs, as they rely on continuous learning from user interactions to enhance personalization.

Future enhancements of the system include expanding its scope by integrating additional content formats, such as audiobooks and magazines, to cater to a broader range of user needs. Collaborative filtering techniques will also be incorporated to complement the content-based recommendation approach, thus improving the quality and diversity of the recommendations. Hybrid recommendation models combining content-based and collaborative filtering approaches have been shown to significantly enhance the recommendation quality. Furthermore, the system will be designed for potential deployment in diverse environments, including libraries, bookstores, and educational institutions, thereby increasing its reach and impact across various sectors.

## 1.7 Limitations

While the proposed book recommendation system offers significant potential, it also faces several key challenges that must be addressed to ensure its effectiveness, fairness, and reliability.

One primary limitation is the system's dependence on diverse and comprehensive data. Without a sufficiently rich dataset, the quality of recommendations may decline, resulting in suggestions that lack relevance or personalization. A prominent issue in this context is the cold start problem, particularly for new users with limited interaction history, which hinders the system's ability to generate accurate recommendations early on.

Additionally, the system may struggle when users are vague about their preferences or when those preferences evolve over time. Without regular updates and adaptive learning strategies, the model may fail to capture such shifts, resulting in stale or misaligned recommendations.

Contextual factors, such as a user's mood, intent, or situational needs, remain particularly difficult to model, yet are critical for delivering deeply personalized suggestions. Another key challenge is the popularity bias, a tendency for models to disproportionately recommend widely consumed or mainstream books, thus limiting exposure to niche or diverse content.

This lack of diversity can hinder the discovery of underrepresented titles and reduce overall user satisfaction.

From an ethical standpoint, LLM-based systems may inadvertently reflect and reinforce societal biases embedded in their training data. This can marginalize certain voices, genres, or cultural narratives if not carefully mitigated. Additionally, since such systems often rely on sensitive user information for personalization, maintaining data privacy and compliance with regulations like GDPR is essential to building user trust.

On the technical side, system scalability and latency are important considerations, especially under high user loads or when processing complex queries. Deep learning models often demand substantial computational resources, and performance bottlenecks may arise without efficient infrastructure. Furthermore, the lack of interpretability in LLM-based recommendations, often seen as "black-box" models, can make it difficult to explain why certain books are suggested, thereby reducing transparency and potentially affecting user trust.

These limitations emphasize the need for continuous evaluation, feedback loops, and responsible AI practices to ensure that the system remains robust, inclusive, and user-centered in the long run.

# 2 Literature Review

## 2.1 Study 1: Content/Context-Aware and Semantics-Aware Recommendation Systems

Before the rise of LLMs, recommender systems relied heavily on content and context modeling. De Gemmis et al. [1] introduced a framework for semantics-aware recommendation that utilized ontologies and knowledge bases to model user preferences and item relationships. These techniques aimed to capture implicit user intent and provide more meaningful suggestions, forming a precursor to the language-based semantic modeling enabled by LLMs. Pradeep et al. [2] developed a content-based movie recommendation system leveraging metadata such as genre, director, and cast information. Their model demonstrated how structured data could be used to build user profiles and generate personalized suggestions. These earlier works remain relevant as LLMs essentially automate semantic feature extraction that was previously manual, offering a more scalable and adaptive alternative to conventional methods.

## 2.2 Study 2: Foundational Understanding of Large Language Models (LLMs)

To understand the integration of large language models (LLMs) into recommender systems, it is essential to first explore their foundational concepts and architectures. The Transformer architecture, introduced by Vaswani et al. [3] revolutionized natural language processing by removing recurrence and convolution in favor of multi-head self-attention mechanisms. This innovation laid the groundwork for LLMs by enabling them to model long-range dependencies more effectively. Following this, Schick and Schütze [4] introduced pattern-exploiting training (PET), demonstrating that even relatively small models could perform few-shot learning effectively, highlighting that model architecture and training methodology can often rival sheer scale. Naveed et al. [5] provided a comprehensive survey of LLMs, elaborating on their core components including tokenization, training objectives, and attention mechanisms, while also discussing their limitations such as hallucinations and high computational costs. Collectively, these works provide the architectural and conceptual background required to appreciate the transformative potential of LLMs in downstream tasks like personalized recommendations.

## 2.3 Study 3: Surveys, Tutorials, and General Exploration of LLM-based Recommender Systems

The intersection of LLMs and recommendation systems has garnered considerable attention in recent years. A major contribution to this space is the vision paper by Wang et al. [6], which proposes a next-generation architecture for recommender systems centered around LLMs, outlining potential shifts in scalability, interpretability, and semantic modeling. Similarly, Zhao et al. [7] assess how LLMs impact traditional recommender system pipelines, especially in addressing cold-start problems. Mao et al. [8] offer a broad survey that differentiates between encoder-based and decoder-based architectures in the context of recommendation

tasks, further discussing open challenges like memory consumption and personalization. Wu et al. [9] provide a domain-specific overview by evaluating the use of LLMs across various application areas including e-commerce, education, and entertainment, while highlighting trends such as multimodal recommendations. A tutorial-oriented approach is taken by Li et al. [10] who simplify the application of LLMs in recommendation for novice researchers, using examples to illustrate their integration and utility. Vats et al. [11] contribute an in-depth literature review focused on the performance benefits, ethical challenges, and limitations of LLM-based recommendation systems, raising concerns about biases and the lack of explainability. Javed et al. [12] serve as a bridge between traditional and modern approaches by reviewing content-based and context-aware strategies and illustrating how LLMs build upon and surpass these methods. Ji et al. [13] introduce the GenRec framework, which utilizes generative modeling via LLMs to recommend items based on language prompts, while Friedman et al. [14] delve into the design of conversational recommendation systems, highlighting LLMs' ability to model user interactions and generate human-like dialogue. These works collectively establish a comprehensive understanding of how LLMs are being applied in modern recommender systems and what future developments might look like.

### 2.4 Study 4: Fine-Tuning Strategies and Optimization for LLMs in Recommendations

Deploying LLMs for recommendation systems in production environments often necessitates resource-efficient fine-tuning techniques. Dodge et al. [15] investigate the impact of factors such as weight initialization, early stopping, and data ordering on the fine-tuning process. Their findings emphasize the importance of subtle design decisions in achieving optimal downstream performance. Lv et al. [16] conduct an empirical study on full-parameter fine-tuning techniques tailored for LLMs in recommendation tasks, particularly under computationally constrained settings. They compare various methods including LoRA (Low-Rank Adaptation), prefix tuning, and adapter layers, all of which aim to reduce training costs without significantly compromising performance. Lin et al. [17] further contribute to this conversation by proposing data-efficient fine-tuning strategies that combine contrastive pretraining with lightweight parameter adaptation, which is especially useful when labeled recommendation data is scarce. These approaches make LLM deployment more feasible in real-world recommendation scenarios where resource constraints and latency requirements are non-trivial

### 2.5 Study 5: Direct Applications of LLMs in Recommendation Tasks

Recent advances have seen LLMs being used directly within recommendation pipelines, either to enhance existing systems or to create entirely new frameworks. Acharya et al. [18] designed a system where LLMs generate descriptive summaries for items in cold-start scenarios, leading to improved recommendation relevance and user engagement. Similarly, Tekle et al. [19] explored LLMs for summarizing song lyrics and associated metadata, enabling more contextually accurate music recommendations. Their results demonstrated that language-based summarization outperformed traditional audio-based recommendation methods in several user

satisfaction metrics. Zhang et al. [20] however, offer a critical perspective on the limitations of using LLMs as standalone recommender systems. They argue that while LLMs excel in generating diverse and plausible content, they often struggle with precision-based ranking unless supplemented with user interaction data or external retrieval modules. These case studies illustrate the diverse ways in which LLMs are currently being integrated into recommendation systems and emphasize both their strengths and limitations.

The literature reveals a rapidly evolving synergy between large language models and recommender systems. Foundational works on LLMs have established the architectural and algorithmic basis, while surveys and application-specific research demonstrate their vast potential in personalization, semantic modeling, and conversational engagement. Efficient fine-tuning strategies are enabling the deployment of LLMs even in resource-constrained environments, making them practical for real-world applications. Moreover, traditional semantics-aware recommendation strategies continue to provide valuable insights that are now being generalized through LLMs. As the field progresses, future work must address critical challenges such as model explainability, bias mitigation, efficient scaling, and hybridization with structured data. These directions are vital for the sustainable and ethical development of LLM-powered recommendation systems.

# 3. Methodology

## 3.1 Development Methodology

The development methodology for this project is centered around a systematic and iterative approach, focusing on the exploration and evaluation of various text vectorization techniques and deep learning-based recommendation strategies. The process is designed to be both experimental and adaptive, allowing for continuous refinement based on observed outcomes and performance metrics.

The initial phase involves a comprehensive survey of Natural Language Processing (NLP) techniques for representing textual data in numerical form. This includes traditional vectorization methods such as Bag of Words (BoW) and TF-IDF, as well as more advanced techniques like word embeddings (e.g., Word2Vec) and contextualized embeddings generated by transformer-based models such as BERT. Each method will be assessed based on its ability to capture the semantic richness and contextual nuances of book metadata, including titles, descriptions, genres, and user reviews.

Following the selection of appropriate vectorization methods, the next phase involves integrating these representations into a recommendation pipeline. Multiple deep learning text vectorization techniques will be considered. Emphasis will be placed on fine-tuning pre-trained language models to enhance similarity detection between books and user preferences.

Throughout the methodology, special consideration will be given to handling challenges such as the cold start problem, data sparsity, and bias mitigation. The development process will also be aligned with best practices in reproducible research, including the use of version control, systematic documentation, and experiment tracking tools.

Ultimately, this methodology aims to identify the most effective combination of text vectorization and learning approaches to build a robust, scalable, and user-personalized book recommendation system powered by state-of-the-art AI.

### 3.1.1 Phase 1

In this phase, we collected a comprehensive dataset containing book metadata such as titles, authors, genres, descriptions, and ratings, primarily using the Google Books API. The data was then cleaned by removing duplicates, handling missing values, and standardizing text through tokenization, stopword removal, and lemmatization.

Basic Exploratory Data Analysis (EDA) was conducted to understand trends, distributions, and potential data issues. Following this, we began experimenting with foundational NLP-based text vectorization techniques, including:

Bag of Words (BoW) for word frequency representation, Bag of N-grams to capture word sequences, and TF-IDF to weigh word importance across the corpus.

These methods prepared the dataset for initial modeling and laid the foundation for more advanced approaches in the upcoming phases.

#### 3.1.1.1 Primary Data Collection

For our project, we compiled a comprehensive and diverse book dataset using the Google Books API, which allowed us to retrieve metadata through structured API requests. Each API key enabled up to 1,000 queries, and for every key, we targeted four specific genres to ensure a balanced distribution of content. Over time, we expanded our scope to cover 56 unique genres, including action, crime, thriller, biography, autobiography, sports, technology, and social sciences, among others. This process resulted in a rich dataset comprising 417,234 unique book entries. Each entry includes ten essential attributes: the book's title, author, publisher, publication date, description, category, page count, average rating, total rating count, and language. This methodical and wide-ranging data collection strategy ensured that our dataset was both robust and representative of the broad literary landscape accessible through the Google Books API.

| | Unnamed: 0 | Title | Authors | Publisher | Published Date | Description | Categories | Page Count | Average Rating | Ratings Count | Language |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | Antiques Roadshow Collectibles | Carol Prisant | Workman Publishing | 2003-01-01 | Offers tips on identifying, collecting, and ca... | Antiques & Collectibles | 660.0 | 4.188379 | 0 | en |
| 1 | 1 | Dog Antiques and Collectibles | Patricia Robak | Schiffer Military History Book | 1999 | A remarkable array of accessible and affordabl... | Pets | 0.0 | 4.188379 | 0 | en |
| 2 | 2 | How to Sell Antiques and Collectibles on eBay.... | Dennis L. Prince, Lynn Dralle | McGraw Hill Professional | 2004-11-15 | Dennis Prince teams up with antique and collec... | Business & Economics | 257.0 | 4.188379 | 0 | en |
| 3 | 3 | Buying & Selling Antiques & Collectibl | Don Bingham, Joan Bingham | Tuttle Publishing | 2012-08-28 | Written for both beginners and experienced col... | Antiques & Collectibles | 254.0 | 4.188379 | 0 | en |
| 4 | 4 | Maloney's Antiques and Collectibles Resource D... | David J. Maloney, Jr. | Antique Trader | 1995-08 | The singular resource that contains contact in... | Antiques & Collectibles | 526.0 | 4.188379 | 0 | en |

*Figure 1 The dataset structure.*

#### 3.1.1.2 Data Preprocessing

Data preprocessing played a crucial role in preparing the collected book data for effective analysis and model training. The raw data extracted from the Google Books API contained inconsistencies such as missing values, duplicate entries, irrelevant records, and variations in formatting. To ensure data quality, we began by removing duplicate and incomplete entries, followed by standardizing text fields such as titles, authors, and descriptions through lowercasing, punctuation removal, and trimming unnecessary whitespace. Categorical data like genres were normalized to maintain consistency across entries. For natural language processing tasks, we further cleaned the textual content by removing stopwords and applying tokenization techniques. This preprocessing phase helped in reducing noise, improving the quality of the textual data, and laying a clean foundation for the application of vectorization techniques and model development.

##### 3.1.1.2.1 Data Cleaning

During the initial phase of data collection, the dataset contained several imperfections that required careful preprocessing. There were missing values, duplicate entries, and redundant stopwords within the description fields, all of which could negatively impact the performance of our recommendation model. To address these issues, we utilized the Pandas library for efficient data cleaning. We removed duplicate records using the drop_duplicates () function to ensure that each book entry was distinct. For missing values, we applied the
fillna () method across most columns to maintain dataset completeness, while deliberately leaving critical fields such as title, page count, rating count, and language unchanged to avoid compromising data accuracy. Additionally, we cleaned the textual data by eliminating common stopwords that could dilute the relevance of book descriptions. After these preprocessing steps, the dataset was refined and reduced to 296,816 unique entries, establishing a clean and reliable foundation for subsequent vectorization and modelling.

##### 3.1.1.2.2 Insights From Books Dataset after Data Cleaning

After refining the Books Dataset through comprehensive data cleaning, we proceeded with Exploratory Data Analysis (EDA) to gain deeper insights into the dataset's structure and characteristics. This phase focused on understanding the distribution of key variables, identifying trends, and detecting any remaining anomalies. By visualizing elements such as genre popularity, author frequency, average ratings, and the distribution of publication years, we were able to uncover patterns that informed our decisions for model selection and feature engineering. EDA served as a crucial step in shaping the direction of our recommendation strategy, ensuring that subsequent modeling efforts were both data-driven and contextually grounded.

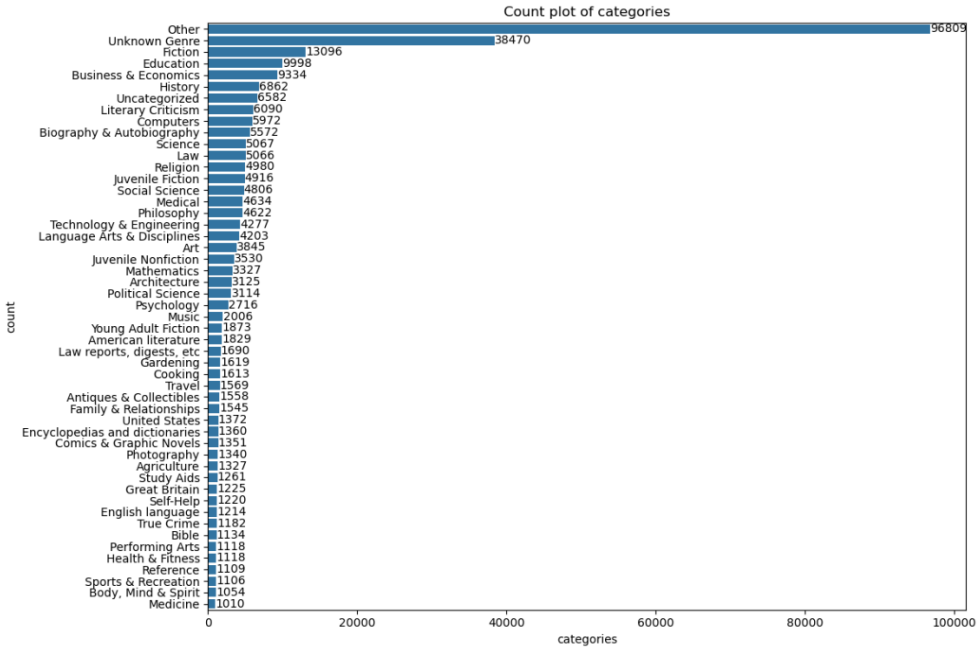



*Figure 2: The dataset category analysis*

One of the most striking observations during our exploratory data analysis was the significant number of books categorized under vague labels such as "Other" or "Unknown Genre." This raised concerns about inconsistencies or gaps in the genre classification process. However, among the well-defined categories, "Fiction" emerged as the most prominent, followed closely by "Education" and "Business & Economics." These categories formed the core of the dataset, while most remaining genres were represented by relatively fewer books. This imbalance illustrated a typical long-tail distribution, where a few dominant genres overshadow a broad range of niche categories. Recognizing this trend provided valuable direction for refining our data and informed how we might balance the recommendations across popular and underrepresented genres.

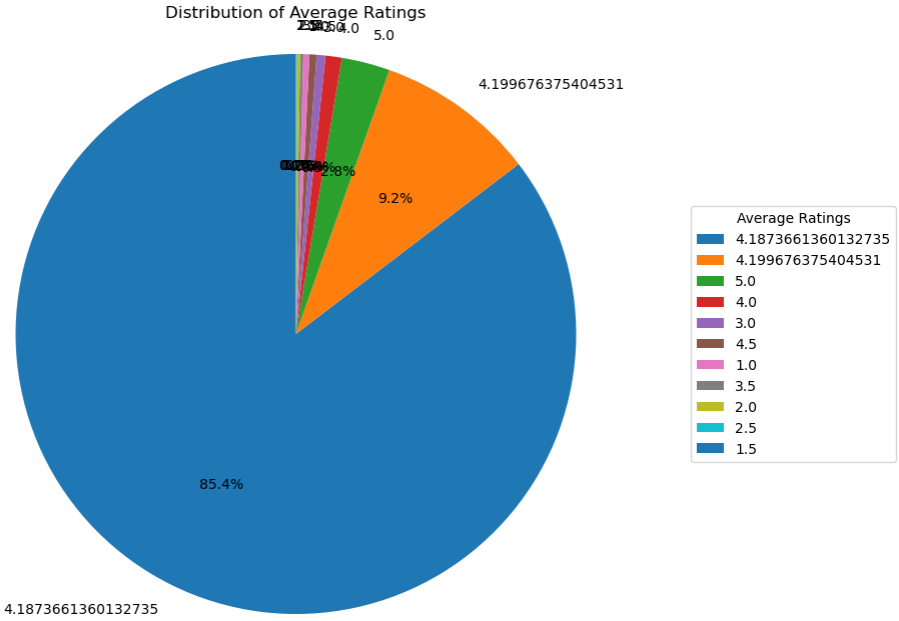



*Figure 3 : Dataset ratings analysis*

To visualize how readers rated the books in our dataset, we created a pie chart that breaks down the distribution of average ratings. Each segment represents a distinct weighted average rating and indicates the proportion of books that fall into that range. What stands out immediately is that a substantial 85.4% of the books have an average rating close to 4.19 (more precisely, 4.1873661360132735), while another notable slice, around 9.2%, clusters tightly around 4.199676375404531. This suggests that most books are receiving consistently high marks, reflecting a general trend of favorable reviews. Whether due to genuinely high-quality content or rating behavior patterns, this narrow spread of strong ratings paints a picture of broadly positive reception throughout the dataset.

### 3.1.1.3 Model Building

Before diving into building the model, we followed a structured, flowchart-based approach to guide the design of our book recommendation system. Our priority was to explore and evaluate various Natural Language Processing (NLP) techniques to determine which would work best for our needs. We experimented with several popular text vectorization methods, including Bag-of-Words, Bag-of-n-grams, and TF-IDF. Each method was tested for its ability to effectively capture the essence of book descriptions and user preferences. This exploratory phase helped us identify the most promising strategy to fuel accurate and meaningful book recommendations as we moved toward model development.

#### 3.1.1.3.1 BAG OF WORDS

The Bag of Words (BoW) method is a classic yet powerful approach in Natural Language Processing (NLP) that we used to convert raw text into numerical data that a machine learning model can understand. BoW works by treating text, such as book descriptions, as a collection of individual words, ignoring grammar and word order, and focusing purely on word frequency.
Here is how we implemented BoW in our project:
Text Preprocessing: We began by cleaning the text, converting everything to lowercase for consistency, removing punctuation and special characters, and filtering out stop words like "the," "is," and "and," which do not add much value. We then tokenized the text, breaking it down into individual words.
Building the Vocabulary: After preprocessing, we scanned through the entire dataset to compile a vocabulary, a complete list of unique words across all documents. This vocabulary became the foundation for transforming the text into numerical form.
Vector Representation: Each book description was then represented as a numerical vector. Each position in the vector corresponded to a word from our vocabulary, and the value at that position indicated how many times that word appeared in the description.
This method gave us a simple but effective way to convert unstructured text into structured numerical data for our recommendation model.

##### 3.1.1.3.1.1 Bag of Words Model Inputs

In our Bag of Words (BoW) setup, we fine-tuned the approach to suit the unique structure of our dataset by analyzing the title and description fields separately. Using CountVectorizer, we transformed text into numerical vectors, limiting the title field to 2,000 unique features and the description field to 3,000 features to manage dimensionality and computational efficiency. Alongside these, we also considered the category of each book to enrich the context. For generating recommendations, we relied on cosine similarity, a technique that quantifies how similar two vectors are in terms of their orientation. To emphasize the strong signaling power of a book's title, we assigned it a 70% weight, while the description contributed 30% to the final similarity score. This weighted blend allowed us to capture both the crisp impact of titles and the deeper context within descriptions, producing more refined and personalized book recommendations.

### 3.1.1.3.1.2 Output

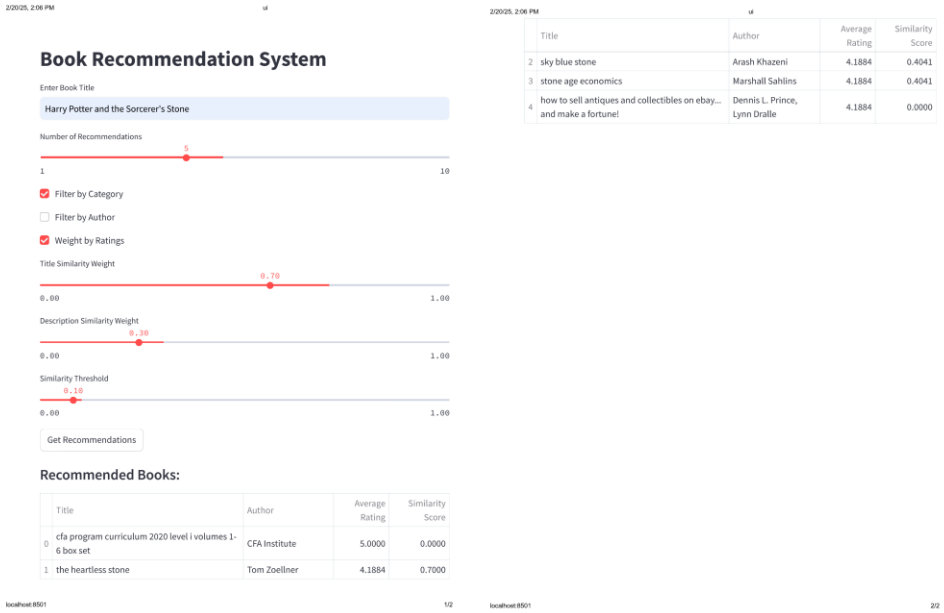



*Figure 4: Output of Bag of words*

### 3.1.1.3.1.3 Output Observation

When we applied the Bag-of-Words (BoW) technique in our book recommendation system, the outcome proved underwhelming. The recommendations it produced often lacked relevance and failed to align with user preferences. This shortfall stems from BoW's fundamental limitation, it focuses purely on word frequency without capturing the context or semantic relationships between words. As a result, the system struggled to understand the deeper meaning behind book titles and descriptions, leading to suggestions that felt generic or off-base. This inability to interpret nuanced language and user intent ultimately limited the effectiveness of the BoW-based recommendation model.

### 3.1.1.3.2 Bag of N-Grams

The Bag of N-Grams model builds on the foundation of the Bag of Words (BoW) method by adding a touch of word order and context through sequences of words called n-grams. Instead of treating each word independently, this model captures combinations like bigrams (pairs of words) or trigrams (three-word sequences), allowing it to better understand how words work together.

Here is how we implemented it in our project:

i. **Text Preprocessing:** Just like in BoW, we started by cleaning the text, converting everything to lowercase, removing punctuation and special characters, and filtering out stop words like “the” and “is.” Then we tokenized the text into individual words.

ii. **Generating N-Grams:** We selected values for n (such as 2 for bigrams or 3 for trigrams) and extracted the corresponding word combinations from the text by sliding a window across it.

iii. **Building the Vocabulary:** Next, we compiled a dictionary of all unique n-grams found across our dataset.

iv. **Vector Representation:** Finally, each document was transformed into a vector based on how often each n-gram occurred, enabling us to numerically compare texts.

This method gave our system a better grasp of context and phrase structure than basic BoW, setting the stage for more refined recommendations.

#### 3.1.1.3.2.1 Bag of N-Grams Model Inputs

In our implementation, we went beyond the traditional Bag of Words model by integrating the Bag of N-Grams technique to capture richer context from the text. For the title and category fields, we used a text vectorizer configured with an n-gram range of (1, 2), enabling it to consider both individual words and two-word combinations. We also set a high feature limit of 200,000 to retain as much meaningful information as possible. The description field, on the other hand, was processed with a vectorizer set to an n-gram range of (2, 3), emphasizing phrases and three-word patterns for deeper semantic cues.

Our model’s key inputs included the title, description, and category of each book. To generate recommendations, we calculated cosine similarity scores between books. We assigned 70% weight to the title, acknowledging its strong influence in capturing the essence of a book, and 30% to the description, allowing its contextual richness to complement the final similarity score. This weighted combination helped us strike a balance between precision and context in the recommendations.

### 3.1.1.3.2.2 Bag of N-gram Output

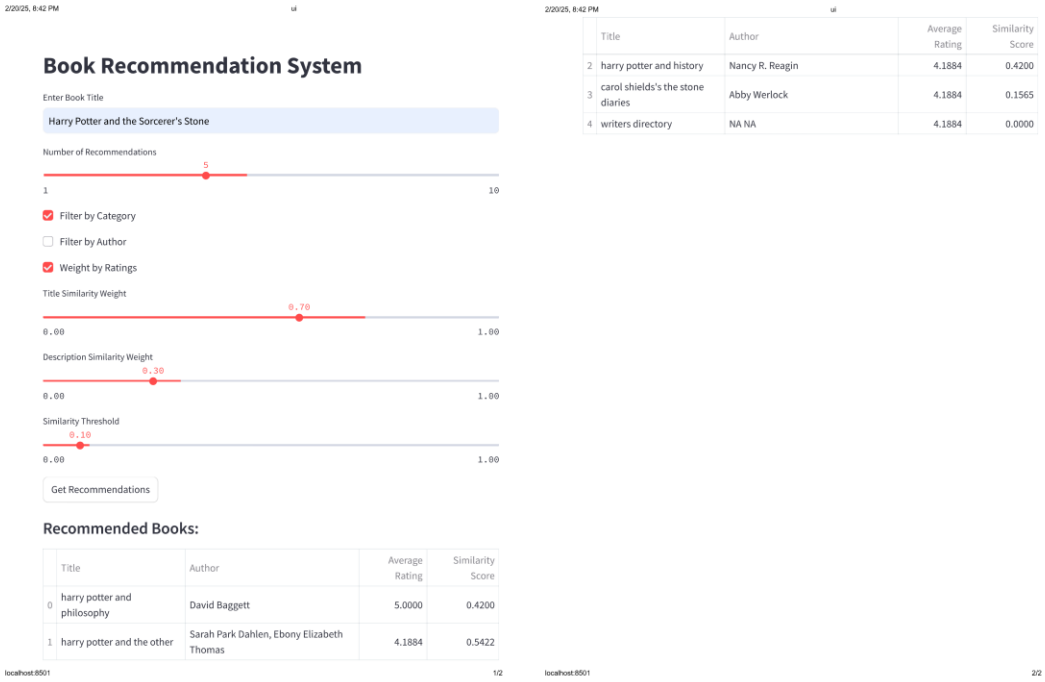



*Figure 5: Output of bag of ngrams*

### 3.1.1.3.2.3 Observation of Output

When we tested the Bag of N-Grams technique in our book recommendation system, the results were underwhelming. While it offered a slight improvement over the basic Bag of Words model by capturing short word sequences, it still struggled to grasp the deeper context and meaning behind the text. The recommendations it generated were only partially relevant and often missed the mark in understanding user preferences. Since this method focuses primarily on the frequency of word combinations without accounting for the semantic relationships between them, it **could not** fully capture the intent or themes of the books. As a result, the suggestions felt somewhat generic and lacked the nuance needed for truly personalized recommendations.

### 3.1.1.3.3 TFIDF

TF-IDF, short for Term Frequency-Inverse Document Frequency, is a powerful refinement in text analysis that elevates how we interpret and weigh words in Natural Language Processing (NLP). Unlike the basic Bag of Words model that treats every word equally, TF-IDF adds nuance by balancing how frequently a word appears in a single document with how rare it is across the entire dataset. This dual scoring system works in two parts: Term Frequency (TF)

measures how often a word shows up in a document, while Inverse Document Frequency (IDF) reduces the importance of words that occur in nearly every document. By multiplying these two values, TF-IDF helps spotlight words that are truly meaningful in a specific context but not overly common across the board. This makes it particularly useful in applications like search engines and recommendation systems, where understanding the unique fingerprint of a document or description can make all the difference.

$$Tf = (\frac{frequency\ \ in\ the\ Doc}{total\ words\ in\ the\ doc})$$

$$Idf = \log\left(\frac{total\ docs}{docs\ having\ a\ term}\right)$$

#### 3.1.1.3.3.1 TFIDF Model Building Input

For our TF-IDF setup, we focused on extracting deeper insights from the title, description, and category fields of each book. The title was vectorized using a TF-IDF vectorizer configured with a feature limit of 200,000 and an n-gram range of (1,2), allowing us to capture both individual words and meaningful two-word phrases. The description field, rich with detail, was given an even broader setup, a TF-IDF vectorizer with a 300,000-feature cap and an n-gram range of (2,3), helping us catch more nuanced phrases and context. The category field also played a supporting role in this process. To generate recommendations, we relied on cosine similarity scores, which allowed us to compare books based on how similar their TF-IDF-weighted textual content was. This method helped us go beyond simple word matching and start tapping into the actual relevance and uniqueness of the book content.

#### 3.1.1.3.3.2 TFIDF Output

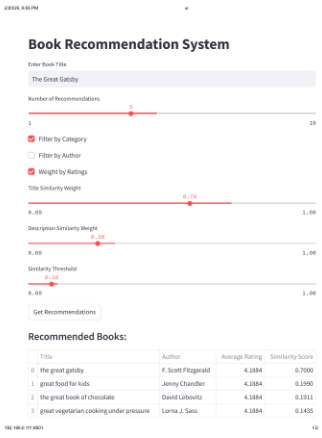



*Figure 6: The output of tf-idf*

#### 3.1.1.3.3.3 TFIDF output Observation:

The TF-IDF approach proved to be a smooth operator, fast, efficient, and well-suited for quick-turnaround text-crunching tasks. It shines when it comes to identifying which words matter within a document by tuning out the noise of frequently used but less informative terms. However, it comes with its own trade-offs. One of the big challenges is the sheer size of the feature space it generates, which can be computationally expensive to manage and scale. While TF-IDF is solid for highlighting relevant terms, it does not capture the underlying meaning or semantic relationships between words. That limitation can hold it back from fully understanding the context or nuance behind a piece of text, something that is crucial when trying to make genuinely relevant book recommendations.

## 3.1.2 Phase 2

In Phase 1, we explored traditional NLP techniques, Bag of Words, Bag of N-Grams, and TF-IDF, to power our book recommendation system. While these methods offered a solid foundation, the recommendations they generated often missed the mark. The core issue? They could not capture the semantic meaning of the book descriptions. These approaches focused heavily on word frequency and local context, but lacked the ability to truly understand the deeper relationships between words and concepts.

To address this gap, Phase 2 marked our shift toward deep learning-based solutions. Here, we experimented with Word2Vec and BERT embeddings, both of which are designed to capture the semantic essence of text. These models move beyond counting words, they understand meaning, context, and nuance. By embedding book descriptions into dense vector representations that preserve semantic relationships, we aimed to produce smarter, more relevant, and context-aware book recommendations.

This phase laid the groundwork for significantly improving the quality and accuracy of our system, pushing us closer to delivering recommendations that click with the users.

### 3.1.2.1 Word2Vec

In Phase 1, we explored natural language processing (NLP) techniques for recommending books, but the results fell short because they failed to fully grasp the deeper meanings within book descriptions. To address this limitation, Phase 2 shifted focus to deep learning methods. Specifically, we experimented with Word2Vec and BERT embeddings to better capture the semantic essence of book descriptions, aiming to improve the quality of our recommendations.

**Word2Vec: Turning Words into Meaningful Vectors**
Word2Vec is a clever neural network-based approach that transforms words into dense vectors within a continuous space. Unlike older methods like one-hot encoding, which create sparse

and disconnected representations, Word2Vec learns from vast text collections to position words with similar meanings closer together. The core idea is simple yet powerful: words that appear in similar contexts likely share similar meanings. Word2Vec offers two key architectures:

**Continuous Bag of Words (CBOW):**
This predicts a target word using the words around it, making it fast and efficient for large datasets.

**Skip-Gram:** This flips the process, starting with a target word and predicting its surrounding context. It shines when dealing with less common words.

By mapping words into this vector space, Word2Vec uncovers relationships like analogies between them, boosting tasks like text classification, sentiment analysis, and even translation. However, it is not perfect. It demands hefty datasets to work well and struggles with words it has not seen before or those with multiple meanings.

#### 3.1.2.1.1 Word to Vector Model Building

For our project, we trained a Word2Vec model using the Google Books dataset, opting for the Skip-Gram method to generate vector embeddings. We paid close attention to tuning the model's settings for the best results. Each word was represented as a 200-dimensional vector, striking a balance between detail and efficiency. We adjusted the context window using a size of 5 for book titles and a larger size of 10 for descriptions to suit their unique characteristics. The Skip-Gram approach proved especially helpful for our recommender system, as it excels at handling rare words, which in turn sharpened the accuracy of our suggestions.

#### 3.1.2.1.2 Word 2 Vec Output

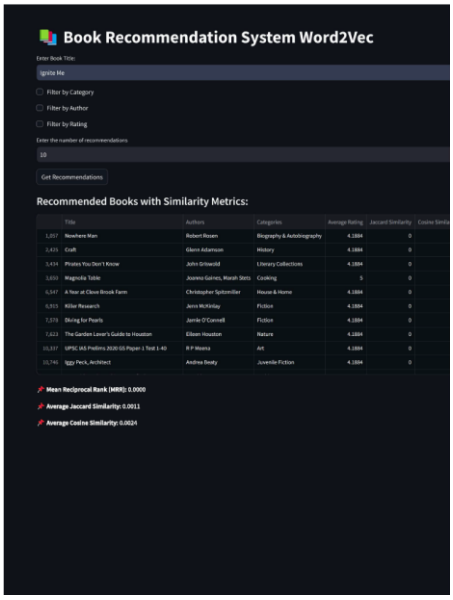



*Figure 7: Output of Word2Vec model*

#### 3.1.2.1.3 Word2Vec Output Observation

Despite our efforts, the Word2Vec model did not quite deliver the results we were aiming for. While it is undeniably more advanced than basic NLP techniques, thanks to its ability to capture semantic relationships between words, the recommendations it produced often mismatched book titles with their genres. This misalignment pointed to a couple of possible weak spots: either our training data was not rich enough, or the embeddings did not fully capture the context needed for accurate matches.

This inconsistency made it clear that Word2Vec, though a step forward, still has its limitations when it comes to nuanced tasks like book recommendations. It is effective at mapping similar words close together in vector space, but it lacks the depth to understand long-range dependencies, sentence structure, and contextual shifts, all of which are critical in truly understanding a book's description and genre.

This realization steered us toward trying even more powerful embedding models like BERT, which are built to overcome exactly these challenges.

### 3.1.2.2 BERT (Bidirectional Encoder Representations from Transformers)

Next, we turned to BERT (Bidirectional Encoder Representations from Transformers), a game-changer in NLP developed by Google. Unlike traditional models that read text in one direction or with limited context, BERT looks both ways left and right giving it a richer understanding of how words fit together. Take the word "bank," for instance: BERT can tell whether it means a financial institution or a river's edge based on the surrounding words.
BERT relies on the Transformer architecture, which uses a mechanism called self-attention to weigh the importance of every word in a sentence. It is pre-trained on enormous datasets with two clever tasks:

- **Masked Language Modelling (MLM):** It hides random words in a sentence and learns to guess them.
- **Next Sentence Prediction (NSP):** It figures out how two sentences connect.

Once pre-trained, BERT can be fine-tuned for all sorts of NLP jobs, from classifying text to answering questions. Unlike Word2Vec's fixed word embeddings, BERT's representations shift depending on context, making it far more adaptable. That said, it is a heavyweight, it needs serious computing power, though lighter versions like Distil BERT and ALBERT help ease that burden.

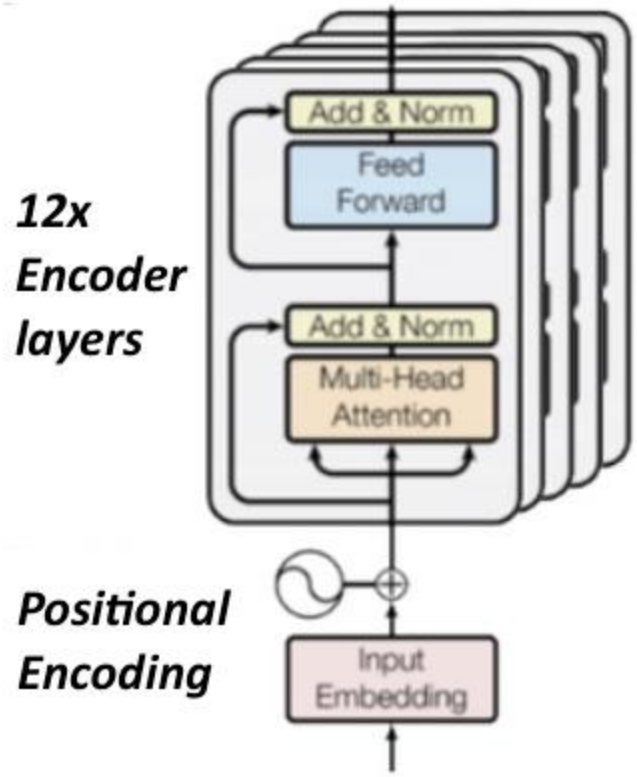



*Figure 8: Bert model used*

At the heart of BERT lies a powerful attention mechanism that enables the model to focus on the most relevant parts of a sentence. It achieves this using three key vectors, Query, Key, and Value, which work together to calculate attention scores that determine how much focus each word should receive in relation to others. These scores are then normalized using a softmax function and used to weight the Value vectors, emphasizing the most meaningful words in the context of the sentence. What sets BERT apart is its use of multi-head attention, where multiple attention "heads" run in parallel, each capturing different aspects of the text such as grammar, relationships between words, or semantic meaning. The outputs from these heads are then combined to form a rich, nuanced representation of the input. This makes BERT exceptionally good at understanding complex and context-dependent language, even across long passages of text, an ability that is crucial for tasks like generating accurate and meaningful book recommendations.

#### 3.1.2.2.1 Attention Mechanism overview

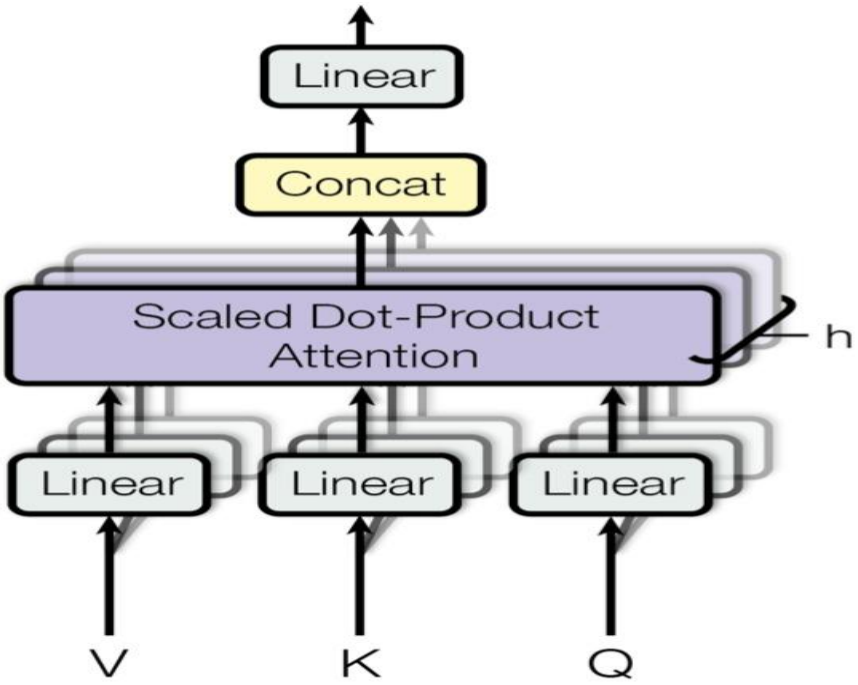



*Figure 9: Multiheaded Attention*

In modern machine learning, particularly within transformer-based models like BERT, the attention mechanism plays a pivotal role. Rather than treating every word or phrase in a sentence with equal importance, this mechanism smartly assigns weights to each token based on how relevant it is to the task or context at hand. By focusing on the most significant parts of the text, the model gains a sharper ability to uncover relationships and meanings, making it a cornerstone of advanced natural language processing.

$$attention(Q, K, V) = softmax\left(\frac{QK^t}{\sqrt{\{D_k\}}}\right)V$$

The multi-head attention mechanism, a key feature of transformer models like BERT, allows the system to process text in a dynamic and multifaceted way. It is designed to spotlight different aspects of the input simultaneously, ensuring a deeper understanding of the content. Here is how it works in detail:

i. **Inputs: Query, Key, and Value**
   The process starts with three critical elements for each token in the sequence: Query (Q), Key (K), and Value (V). These are created by transforming the input embeddings through linear operations. Think of them as different lenses: the Query defines what

the model is searching for, the Key pinpoints which tokens matter, and the Value holds the actual content to focus on.

ii. **Scaled Dot-Product Attention**

Each "attention head" calculates a score by comparing the Query and Key vectors using a scaled dot-product. To keep things stable, this score is adjusted by the square root of the vector's dimensionality. These scores are then normalized using a soft max function, turning them into probabilities. These probabilities determine how much weight to give the Value vectors, producing a weighted blend that emphasizes the most relevant tokens based on their context.

iii. **Multi head attention in action**

The system does not stop at one perspective. Multiple attention heads represented as "h" work independently, each zeroing in on different linguistic details. One head might pick up on sentence structure, while another tunes into meaning or tone. Once each head has done its job, their outputs are combined (shown as "Concat") and refined through a final linear transformation. This step weaves together a rich, unified representation of the text.

$$Multihead(Q, K, V) = Concat(h_1, h_2 \dots h_n)W^0$$

$$where\ hi = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

iv. **Why It Works So Well**

Parallel Processing: With multiple heads running at once, the model can tackle various angles of the text in one go, saving time and boosting efficiency.

Deeper Contextual Insight: This approach excels at capturing subtle word relationships, even across lengthy sequences, making it ideal for complex language tasks.

The attention mechanism, especially in its multi-head form, is what powers transformer models to excel at understanding text. For our book recommendation system, this ability to focus on what truly matters in descriptions whether it is themes, tones, or specific phrases helps us move beyond surface-level analysis. By integrating this into our approach, we are building a system that does not just read text but truly gets it, paving the way for more accurate and meaningful recommendations.

Positional Encoding in BERT:

$$PE\ (pos,2i) = sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE\ (pos,\ 2i+1) = cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Where:

- Pos represents the position of a time step in the sequence.
- d is the embedding dimension**.**

**The** sine and cosine functions allow the model to learn relative positional information efficiently.

#### 3.1.2.2.2 Faiss Library

To handle the dense vectors from our embeddings efficiently, we brought in FAISS (Facebook AI Similarity Search), a tool built by Meta's AI team for fast and scalable similarity searches. FAISS is designed to manage huge datasets beyond what RAM alone can hold using metrics like Euclidean distance or cosine similarity to compare vectors accurately. It is turbocharged with GPU support and smart indexing tricks, like inverted files and quantization, to keep searches quick and memory-light. In our project, FAISS streamlined the process of finding similar books based on their embeddings, making it a vital piece of our recommendation engine.

**How FAISS Works**
FAISS starts with high-dimensional vectors, builds an optimized index, and uses a quantizer to shrink them down without losing too much detail. An inverted file system organizes everything into clusters for fast retrieval, while GPU power ensures it can scale up as needed. This setup let us efficiently match books based on their semantic profiles.

#### 3.1.2.2.3 BERT Model Building

To harness BERT for our book recommendation system, we carefully prepared the input data by blending text and numeric features into a cohesive pipeline. For the text-based columns Title, Description, and Categories, we standardized the data by converting everything to lowercase and ensuring a uniform string format. Next, we tokenized these columns, tailoring the process to their unique needs: titles were capped at a maximum length of 128 tokens, descriptions at 256, and categories at 64. The BERT tokenizer handled padding and truncation to keep things consistent, delivering the results as Py Torch tensors ready for the model. We set the training to run for 3 epochs, striking a balance between learning and efficiency. On the numeric side, features like Page Count, Average Rating, and Ratings Count were normalized using a Standard Scaler to ensure even distributions. The Language column, meanwhile, was transformed into numbers using a Label Encoder. To manage and retrieve book data efficiently, we built an SQLite database for storage, paired with the FAISS library to handle similarity searches across our large dataset. Our recommendation system pulls suggestions based on a book's title, description, and category, leveraging three FAISS indexing methods IVF Flat,

HNSW, and Flat L2 to test their performance. Each method was applied separately during training and recommendation phases, with similarity scores calculated independently to compare how well they delivered relevant results.

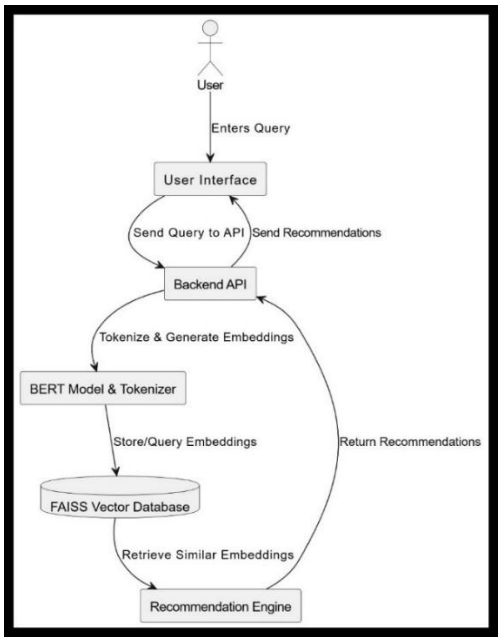



*Figure 10: Bert system architecture*

The recommendation system's workflow, illustrated in a flowchart, ties BERT and FAISS into a seamless process. It starts with a user entering a query through a simple interface. The backend API takes this input, tokenizes it, and feeds it into the BERT model to create embeddings compact numerical representations of the text. These embeddings are stored in a FAISS vector database, which quickly retrieves similar entries using its powerful search capabilities. The recommendation engine then refines these matches into personalized suggestions, sending them back to the user via the API. This setup blends cutting-edge machine learning with efficient vector search to produce high-quality recommendations.

### 3.1.2.2.4 BERT Output Recommendations

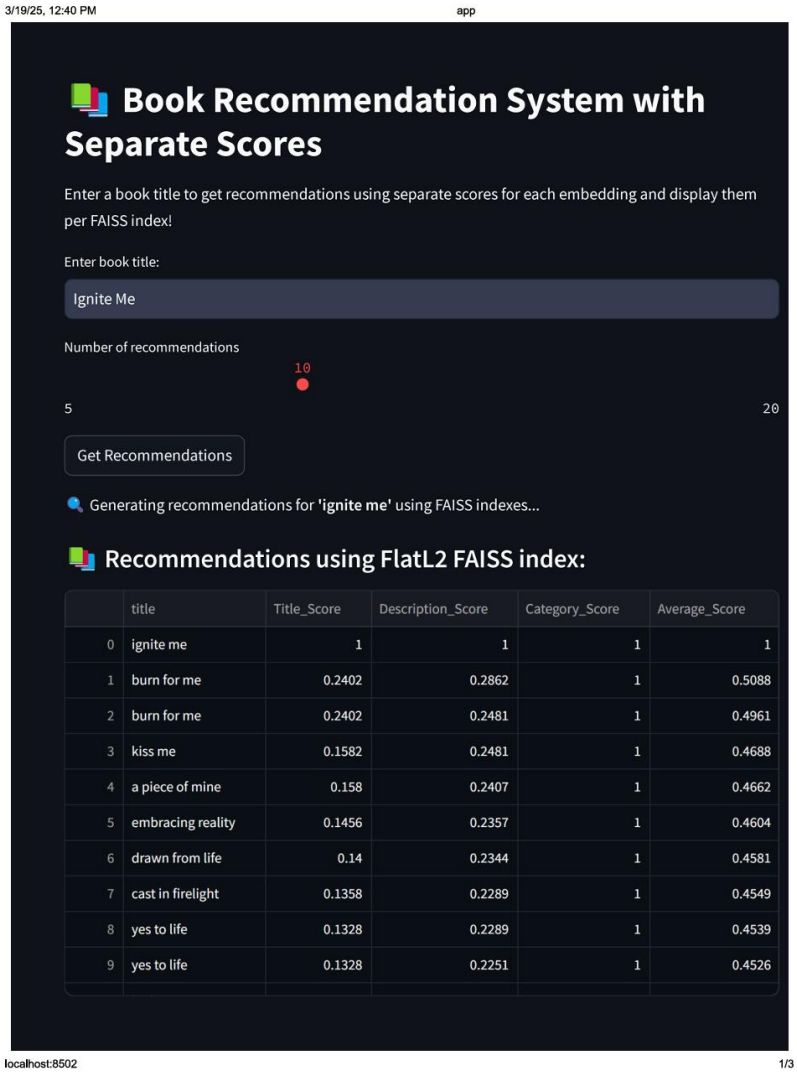
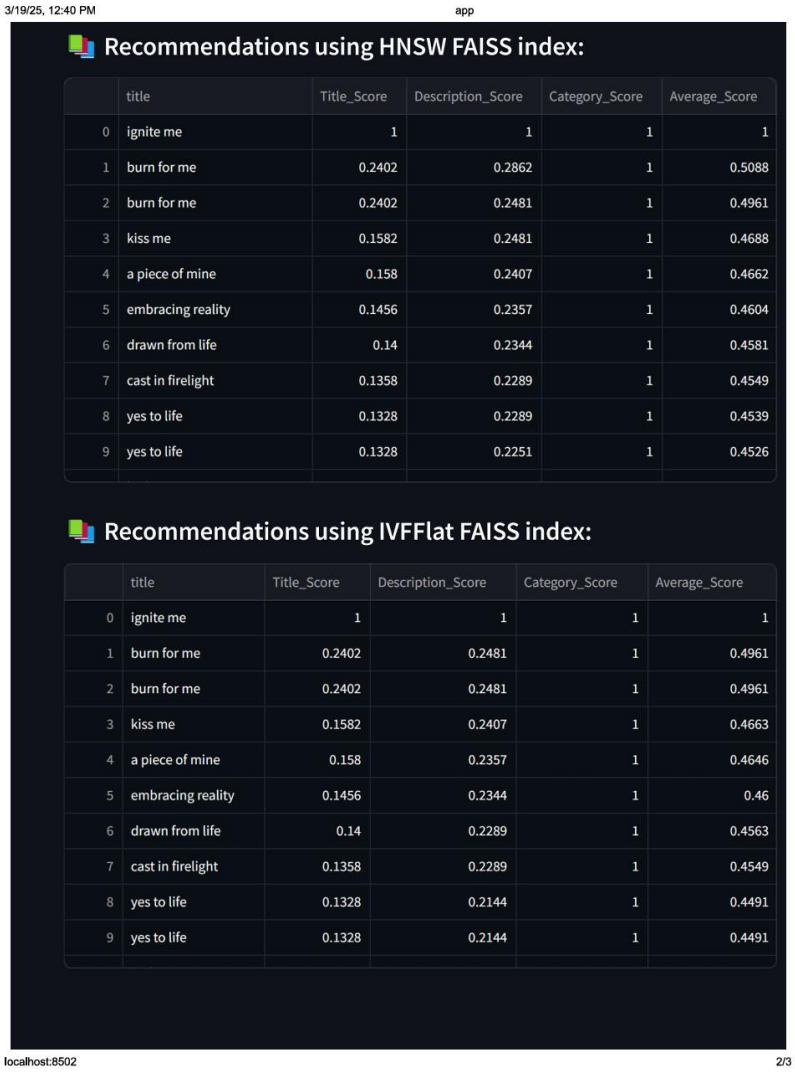



*Figure 11: Bert system output*




| | title | Title_Score | Description_Score | Category_Score | Average_Score |
|---|---|---|---|---|---|
| 0 | ignite me | 1 | 1 | 1 | 1 |
| 1 | burn for me | 0.2402 | 0.2862 | 1 | 0.5088 |
| 2 | burn for me | 0.2402 | 0.2481 | 1 | 0.4961 |
| 3 | kiss me | 0.1582 | 0.2481 | 1 | 0.4688 |
| 4 | a piece of mine | 0.158 | 0.2407 | 1 | 0.4662 |
| 5 | embracing reality | 0.1456 | 0.2357 | 1 | 0.4604 |
| 6 | drawn from life | 0.14 | 0.2344 | 1 | 0.4581 |
| 7 | cast in firelight | 0.1358 | 0.2289 | 1 | 0.4549 |
| 8 | yes to life | 0.1328 | 0.2289 | 1 | 0.4539 |
| 9 | yes to life | 0.1328 | 0.2251 | 1 | 0.4526 |

| | title | Title_Score | Description_Score | Category_Score | Average_Score |
|---|---|---|---|---|---|
| 0 | ignite me | 1 | 1 | 1 | 1 |
| 1 | burn for me | 0.2402 | 0.2481 | 1 | 0.4961 |
| 2 | burn for me | 0.2402 | 0.2481 | 1 | 0.4961 |
| 3 | kiss me | 0.1582 | 0.2407 | 1 | 0.4663 |
| 4 | a piece of mine | 0.158 | 0.2357 | 1 | 0.4646 |
| 5 | embracing reality | 0.1456 | 0.2344 | 1 | 0.46 |
| 6 | drawn from life | 0.14 | 0.2289 | 1 | 0.4563 |
| 7 | cast in firelight | 0.1358 | 0.2289 | 1 | 0.4549 |
| 8 | yes to life | 0.1328 | 0.2144 | 1 | 0.4491 |
| 9 | yes to life | 0.1328 | 0.2144 | 1 | 0.4491 |

*Figure 12: Bert system output (next page)*

### 3.1.2.2.5 BERT Output Observation

When we put BERT to the test, it outshone Word2Vec in a big way. Its knack for handling longer descriptions and picking up on contextual connections made a noticeable difference in the quality of recommendations. Unlike Word2Vec, BERT digs deeper into the meaning behind the text, resulting in more relevant suggestions. That said, it is not without flaws it struggles to tailor recommendations to individual preferences in the way traditional data-driven systems can. These figures highlight its precision and reliability in matching books to queries. While it is not perfect for every personalization need, BERT's strength in understanding context makes it a standout choice for our system, setting a high bar for recommendation quality.

### 3.1.3 Phase 3

In Phase 2, our BERT-based system made significant strides in delivering personalized book recommendations with a conversational touch. However, to further enhance semantic understanding and elevate the precision of our recommendations, Phase 3 marks a strategic shift toward leveraging a fine-tuned GPT model. This transition is driven by the goal of embedding deeper context, nuance, and user intent into each interaction, enabling the system to generate more insightful and engaging suggestions. With GPT's powerful language generation capabilities, we aim to craft recommendations that feel more intuitive, human-like, and contextually aware, transforming the user experience from a basic search into a seamless and enjoyable discovery journey.

#### 3.1.3.1 Sentence Transformers

Traditional transformer models like BERT were originally designed for tasks that focus on token-level classification or question answering. While powerful, they are not optimized for comparing or embedding entire sentences. This is where Sentence Transformers come into play.

Developed initially by UKPLab, Sentence Transformers (also known as SBERT) are modified versions of transformer-based architectures that are specially trained to create meaningful sentence embeddings. These embeddings allow us to efficiently compare sentence similarity, perform semantic search, clustering, and other downstream NLP tasks.

The main idea is to fine-tune transformer models using Siamese or triplet networks, so that semantically similar sentences are mapped to nearby points in the vector space. Once trained, you can pass a sentence or even a paragraph and get a dense, fixed-size vector that captures its core meaning.

Sentence Transformers are not just about improved performance, they are also designed for speed and scalability, making them a practical choice for real-world applications like information retrieval or recommendation systems.

##### 3.1.3.1.1 Detailed Overview of all-MiniLM-L6-v2 and Its Role in My Book Recommendation System

Among the many pre-trained models available in the Sentence Transformers library, all-MiniLM-L6-v2 was chosen for its excellent balance of speed, memory efficiency, and semantic understanding. The model is built on the MiniLM architecture, a compact transformer model that includes only six transformer layers with a hidden size of 384. This makes it significantly faster and lighter than larger models like BERT or RoBERTa, yet it delivers comparable performance on most sentence-level tasks.

The all-MiniLM-L6-v2 model is trained using a contrastive learning approach known as multiple negative ranking loss. During training, the model learns to distinguish a relevant sentence pair from many irrelevant ones. This type of training enhances the model's ability to capture subtle semantic relationships, even in varied contexts. The training data includes a diverse mix of datasets such as SNLI, MultiNLI, Quora duplicate questions, and Natural Questions, enabling the model to generalize well across multiple domains. The "all" in its name refers to its broad applicability across question-answer pairs, web text, and conversational data.

In my book recommendation system, all-MiniLM-L6-v2 plays a central role in generating semantic embeddings for various textual features of the books, such as the title, description, and category. Each of these components is processed separately through the model to produce high-dimensional vector embeddings that encapsulate their meaning. These embeddings are then indexed using FAISS (Facebook AI Similarity Search) to build a similarity-based recommendation engine. When a user provides a book title or query, the system retrieves the most semantically similar books based on cosine similarity between embeddings.

This approach allows the system to go beyond keyword matching and deliver more contextually relevant book suggestions. For instance, even if a user's input does not exactly match a book title in the dataset, the semantic similarity between descriptions or genres ensures that thematically related books can still be recommended. This not only improves the recommendation quality but also enhances the user experience by making the system more intuitive and intelligent. Thanks to the speed and efficiency of all-MiniLM-L6-v2, the system remains scalable and responsive, even when processing thousands of book entries in real-time.

**Mathematical explanation of the model architecture**

Sentence transformer

- Model name: all-MiniLM-L6-v2
- Base architecture: MiniLM
- Transformer layers: 6
- Hidden size: 384
- Sentence Embedding: Mean Pooling over token embeddings
- Training Objective: Contrastive Learning

**Positional encoding**

Given sentence $S = [w_1, w_2, \dots w_n]$. Each token is mapped to an embedding vector.
Embedding matrix $E \in R^{V \times d}$
Token Embedding $x_i \in R^d$
Positional embedding $p_i \in R^d$, where V is the vocabulary size and the d is the hidden size.
Each token is mapped to a vector $x_i \in R^d$ and positional encodings $p_i \in R^d$ are added:

$$\boldsymbol{h_i^0 = x_i + p_i}$$

**Transformer layer (repeated 6 times)**

**Multi head Self Attention**: For each attention head h $\in [1, H]$:

$$\boldsymbol{Q} = \boldsymbol{H}^{\{(\boldsymbol{l-1})\}}\boldsymbol{W}_{\boldsymbol{Q}}^{\{(\boldsymbol{h})\}}$$
$$\boldsymbol{K} = \boldsymbol{H}^{\{(\boldsymbol{l-1})\}}\boldsymbol{W}_{\boldsymbol{K}}^{\{(\boldsymbol{h})\}}$$
$$\boldsymbol{V} = \boldsymbol{H}^{\{(\boldsymbol{l-1})\}}\boldsymbol{W}_{\boldsymbol{V}}^{\{(\boldsymbol{h})\}}$$

**Self-Attention:**

$$Attention(Q, K, V) = softmax\left\{\frac{(QK^t)}{\sqrt{D\_k}}\right\}V$$

**Multi head attention:**

$$MHSA\left(H^{\{(l-1)\}}\right) = Concat(head_1, \dots, head_H)W_0$$

**Feed forward network:**

$$FFN(x) = GELU(x\, w_1 + b_1)\mathrm{w}_2 + \mathrm{b}_2$$

**Residual Connections + Layer Norm:**

$$LayerNorm(MHSA(H^{l-1} + H^{l-1})$$
$$H^l = LayerNorm(FFN(H^l) + H^l)$$

**Mean Pooling for sentence embedding:**

After the final transformer layer (6th layer), we have token embeddings $H^6 = [\, h_1, h_2 \dots h_n]$. These are mean pooled.

$$SentenceEmbedding = \frac{1}{n}\sum_{i=1}^{n}(h_i)$$

This gives a fixed-size $s \in R^{384}$ embedding for the entire sentence.

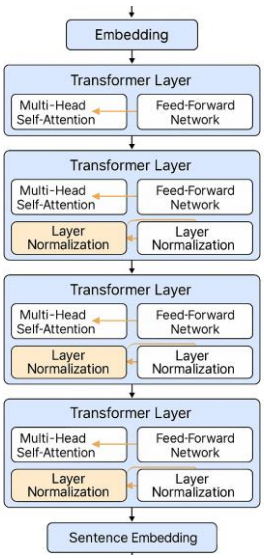



*Figure 13: Sentence Transformer Architecture*

### 3.1.3.2 Decoder-Only Architecture in Transformers

The decoder-only architecture is a streamlined version of the transformer model, focusing solely on generating outputs based on prior inputs, perfect for tasks like text generation, code completion, and dialogue systems. Unlike the original transformer model, which has both an encoder and a decoder (commonly used for translation), the decoder-only model eliminates the encoder and simplifies the design.

At the core of this architecture is a stack of decoder blocks, each equipped with self-attention and feedforward layers. The self-attention mechanism is masked, meaning the model can only attend to previous tokens in the sequence and not future ones. This makes it ideal for autoregressive tasks, where the model predicts the next word one token at a time.

**Each decoder block consists of:**

**Masked Self-Attention:** Prevents the model from "cheating" by looking ahead, allowing it to predict the next word based only on what has come before.

**Feedforward Neural Network**: Adds non-linearity and complexity to better capture patterns in the text.

**Residual Connections and Layer Normalization:** Helps with training stability and deeper learning.

Positional embeddings are added to the input embeddings to give the model a sense of word order since transformers do not have built-in sequence awareness.

The output from the final decoder block is passed through a linear layer and a softmax function to generate a probability distribution over the vocabulary, from which the next word is selected.

Models like GPT-2, GPT-3, and GPT-4 are built on this decoder-only framework, proving that even without an encoder, deep contextual understanding and impressive language generation are possible.

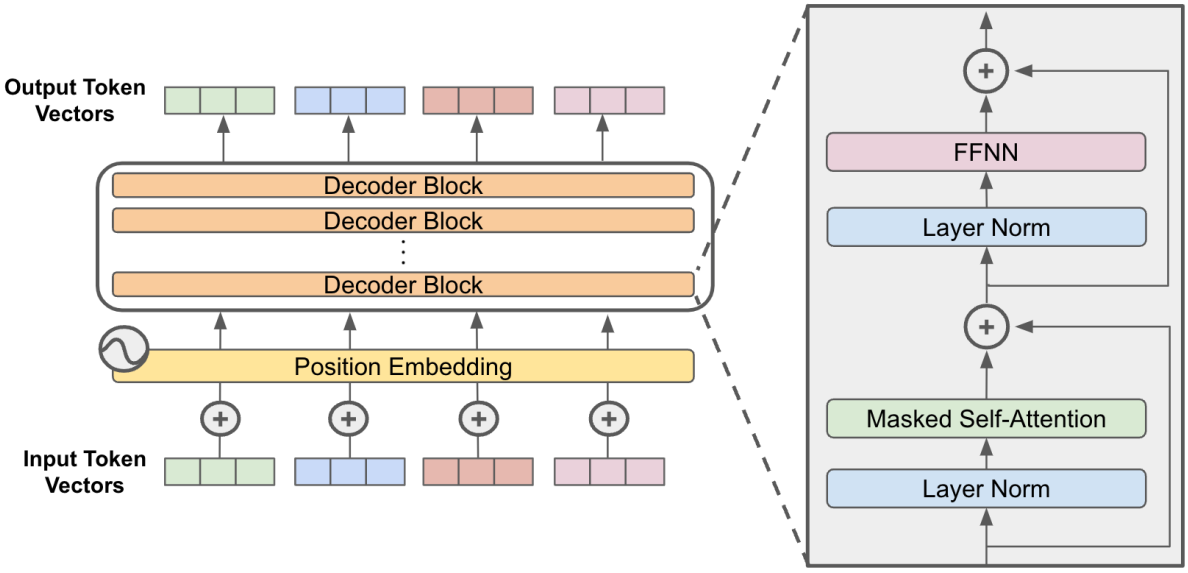



*Figure 14: Decoder only transformer model architecture*

### 3.1.3.3 Falcon 3 1B

Falcon 3-1B is a decoder-only transformer model developed by the Technology Innovation Institute (TII). It is designed to optimize efficiency, scalability, and cost-effectiveness, making it suitable for various natural language processing tasks, including text generation and comprehension. Unlike larger models, Falcon 3B-1 employs an autoregressive approach, wherein each token is predicted based on preceding tokens, which is particularly effective for tasks such as text summarization, conversational systems, and content generation.

The model's architecture is optimized for low memory consumption, allowing it to perform well even in resource-constrained environments. It has been trained on a curated dataset to minimize bias and generate coherent, contextually relevant outputs. Additionally, it incorporates Multi-Query Attention (MQA), a mechanism that improves inference speed while

maintaining output quality.

Although smaller than models such as Falcon 40B, Falcon 3-1B offers a compelling balance between performance and accessibility. Its capacity for fine-tuning further enhances its applicability in domain-specific use cases, rendering it a practical choice for researchers, developers, and enterprises seeking efficient language models.

**Mathematical details of the model being used:**

**Falcon3 1B**

**Input Embedding and Positional Encoding**

- Input tokens: $X = [x_1, x_2, ..., x_n]$
- Token Embedding: $h_i^0 = E_t(x_i) + E_p(i)$
- $E_t \in R^{\{V \times d\}}$ , $E_p \in R^{\{n \times d\}}$ where d = 2048

**Decoder Block – Causal Attention**

- Query: $Q = \{H^{\{l-1\}}\}W_Q$
- Key: $K = \{H^{\{l-1\}}\}W_K$
- Value: $V = \{H^{\{l-1\}}\}W_V$
- $Attention(Q, K, V) = softmax\left\{\frac{(QK^t)}{\sqrt{D\_k}} \times M\right\}V$
- Mask M prevents future tokens from being seen

**Feedforward and Residual Connections**

- LayerNorm → Self-Attn → Residual → LayerNorm → FFN → Residual
- $FFN(x) = GELU(xW_1 + b_1)W_2 + b_2$
- Each of the 24 layers follows this pattern

**Output and Causal Language Modeling**

- Final Output: $H^l = [h_1, h_2, \ldots, h_n]$
- Logits: $logits_i = h_i W^T + b$
- Probability: $P(x_{i+1} \mid x \leq i\}) = softmax(logits_i)$

# 4. System Design and Architecture

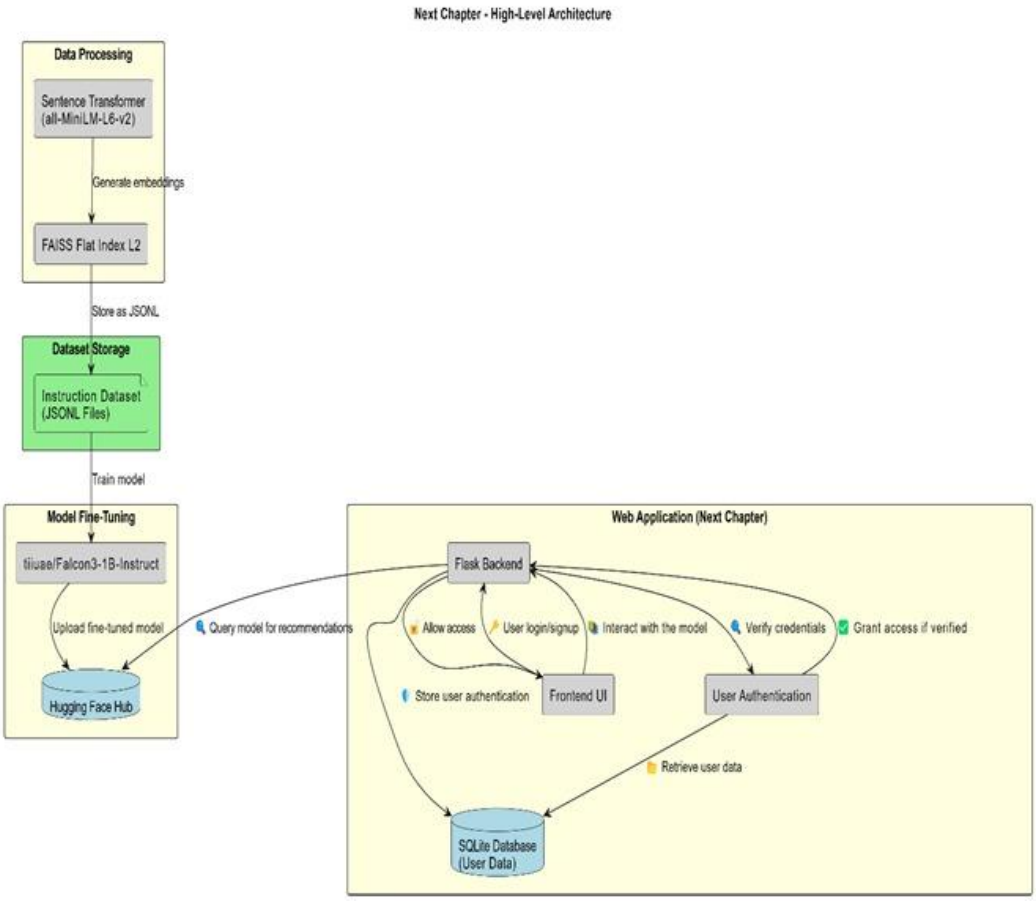



*Figure 15: Final system architecture*

## 4.1 Proposed System Architecture

The proposed system, Next Chapter, is a personalized book recommendation platform that leverages state-of-the-art natural language processing techniques to provide conversational and context-aware suggestions to users. The system architecture is organized into four primary components: Data Processing, Dataset Storage, Model Fine-Tuning, and the Web Application Interface. An overview of the high-level architecture is illustrated in Figure 13.

### 4.1.1 Data Processing

The data processing phase involves transforming raw textual information (e.g., book titles, descriptions, and categories) into dense semantic representations using the all-MiniLM-L6-v2 Sentence Transformer. These representations, or embeddings, are computed to capture the

contextual meaning of the input data. To enable efficient retrieval, the embeddings are indexed using a FAISS Flat Index (L2 distance metric). This index supports rapid similarity searches based on vector proximity. The resulting indexed embeddings are stored in JSON Lines (JSONL) format for subsequent use.

### 4.1.2 Dataset Storage

The processed embeddings are organized into an instruction-style dataset and stored in the JSONL format. This dataset forms the training corpus for fine-tuning the language model. By aligning user instructions with appropriate book responses, the system is trained to simulate conversational recommendation scenarios effectively.

### 4.1.3 Model Fine-Tuning

For generating human-like and contextually relevant recommendations, the system employs Falcon-3B-Instruct, a decoder-only transformer model developed by the Technology Innovation Institute (TII). The model is fine-tuned on the instruction dataset to adapt it specifically to the task of book recommendation. After fine-tuning, the model is hosted on the Hugging Face Hub, enabling remote access and interaction through an API. The Flask backend of the application queries the hosted model in real time to obtain recommendation outputs.

### 4.1.4 Web Application (Next Chapter)

The user-facing layer of the system is implemented as a web application using the Flask microframework. This component manages user interaction, authentication, and communication with the recommendation model.

i. **Frontend UI:** Provides a user-friendly interface for login, signup, and book recommendation queries.

ii. **Flask Backend:** Serves as the central coordinator, handling user requests, querying the model, and managing access control.

iii. **User Authentication Module:** Verifies user credentials and ensures secure access by cross-referencing stored authentication data.

iv. **SQLite Database:** Stores user data and authentication details, and supports retrieval operations during the login and recommendation processes.

v. Upon user login, the system verifies credentials through the authentication module and grants access if validated. The user's query is then passed to the backend, which in turn communicates with the fine-tuned model to retrieve a personalized recommendation. The result is rendered via the frontend interface.

### 4.1.5 End-to-End Workflow

i. Raw book data is transformed into semantic embeddings using a Sentence Transformer.

ii. Embeddings are indexed with FAISS and stored as an instruction dataset.

iii. The instruction dataset is used to fine-tune the Falcon-3B-Instruct model.

iv. The fine-tuned model is deployed to Hugging Face Hub.

v. The Flask web application manages user authentication and interfaces with the model to deliver book recommendations.

vi. This modular architecture ensures scalability, ease of maintenance, and an enhanced user experience through natural language interactions.

## 4.2 Falcon 3 1B Model for recommendation system

To drive the core of our book recommendation system, we fine-tuned the Falcon 3B-1B Instruct model using a domain-specific instruction dataset. The fine-tuning process was meticulously designed to align the model with the unique requirements of personalized, conversational book recommendations. The fine-tuned model is hosted on the Hugging Face Hub, ensuring accessibility and ease of deployment for downstream application integration.

During the training phase, key performance metrics were monitored to evaluate the model's learning progression. The evaluation loss, a primary indicator of model accuracy, decreased from 0.20 to 0.15 by the conclusion of the testing phase, reflecting notable improvements in generalization. Additionally, the model achieved a perplexity score of 5.15, indicating strong predictive confidence in generating coherent and relevant responses.

The training configuration was optimized for both efficiency and performance. A batch size of 32 was selected to maintain a stable computational load, and training was conducted using the AdamW optimizer, known for its adaptability and widespread use in transformer-based architectures. A learning rate of 0.00325 was employed to ensure consistent convergence without oscillation or overshooting.

Training was executed over a single epoch spanning 230,513 steps, with the entire process completed in under 15 hours using an H100 GPU provisioned by Ola Krutrim Cloud. These hyperparameters and infrastructure choices were deliberately selected to facilitate efficient learning while maximizing the model's ability to deliver precise, context-aware, and preference-driven book recommendations.

### 4.2.1 Observation of Output

The deployment of Falcon 3-1B marked a significant advancement in the performance and user-centricity of our book recommendation system. Compared to earlier approaches such as BERT and other deep learning models, Falcon 3-1B demonstrated superior capabilities in capturing semantic nuances and contextual dependencies. Its decoder-only architecture enabled it to process and respond to user queries with heightened coherence and intent awareness, leading to substantially more accurate and contextually relevant recommendations.

One of Falcon's key strengths lies in its ability to manage extended conversational context, a feature that proved crucial in sustaining relevance throughout multi-turn interactions. This capacity allowed the system to maintain alignment with the user's evolving preferences, ensuring that recommendations were not only accurate but also meaningfully personalized.

The transition from BERT-based systems to Falcon 3-1B was more than a technical upgrade; it represented a paradigm shift in user engagement. By leveraging the generative power of GPT-style models and combining it with Falcon's efficiency and fine-tuning adaptability, we crafted a system that delivers intuitive, natural, and personalized suggestions.

The results validated this transition. Falcon 3-1B consistently produced more insightful, contextually aware, and engaging responses, thereby enhancing the overall recommendation experience. This milestone in Phase 3 of our system evolution underscores our commitment to delivering not only precise but also emotionally resonant user interactions, paving the way for a recommendation framework that is both dependable and delightful.

**Table 1: Observations and Summary table**

| Model name | Metric used for recommendation generation and model evaluation | Observation of the recommendation quality |
|---|---|---|
| **Bag Of Words** | Cosine Similarity, Euclidean distance, Mean Reciprocal Rank | Poor quality of recommendations due to lack of context |
| **Bag Of Ngrams** | Cosine Similarity, Euclidean distance, Mean Reciprocal Rank | Having very poor quality of recommendation and high time complexity |
| **Tf – Idf** | Cosine Similarity, Euclidean distance, Mean Reciprocal Rank | Loss of semantic meaning led to poor recommendations |
| **Word2vec** | Cosine similarity between embeddings, Mean Reciprocal Rank | Performed badly for long descriptions |
| **Bert** | Euclidean distance, HNSW, Mean Reciprocal Rank | Gave decent recommendations with semantic meaning, but does not have the text generation capability |
| **Falcon 3 -1B** | FlatL2 (sentence transformer), Perplexity, Loss, Mean Reciprocal Rank | Highly relevant recommendation quality with deep semantic meaning, powerful system with generative abilities. |

# 5. System Development Process and Testing

Creating an effective book recommendation system required a structured approach, blending data preparation, advanced modelling, and user-friendly deployment. Below, we outline the key steps that brought this system to life, designed to deliver accurate and engaging suggestions for book lovers.

## 5.1 Preparing the Data

The foundation of our system lies in the data. We gathered a rich dataset of book details titles, descriptions, categories, authors, and ratings and set to work refining it. This meant cleaning the text by removing inconsistencies, standardizing formats, and ensuring everything was polished and uniform. A well-prepared dataset paved the way for reliable analysis and recommendations down the line.

## 5.2 Crafting Embeddings

Next, we turned the raw text into something the system could understand: embeddings. Using Sentence Transformers, we generated distinct embeddings for titles, descriptions, and categories, capturing their unique meanings in a numerical format. To make retrieval fast and efficient, we stored these embeddings in a FAISS index a tool that excels at handling large-scale similarity searches. This step ensured our system could quickly find and match books based on their core characteristics.

## 5.3 Fine-Tuning with Instruction Data

To tailor the model to our needs, we transformed the book data into an instruction-based JSONL format, setting the stage for fine-tuning. We chose the Falcon-3B-Instruct model and refined it on Ola Krutrim Cloud, powered by the high-performance H100 GPU. This process sharpened the model's ability to interpret user inputs and generate spot-on recommendations, aligning it closely with the nuances of our book dataset.

## 5.4 Deployment

With the model ready, we deployed it for real-world use. First, we uploaded it to the Hugging Face Hub, making it accessible and shareable. Then, we built a Flask-based backend with endpoints to handle recommendation requests smoothly. For the front end, we crafted a Streamlit interface, complete with filters for categories, authors, and ratings, giving users an intuitive way to explore and find their next read.

## 5.5 Testing

To ensure quality, we put the system through rigorous testing. The results were promising: a perplexity score of 5.15 indicated strong predictive confidence, while a testing loss of 1.3 reflected solid accuracy in the model's outputs. These metrics confirmed that our system was on the right track, balancing precision with practical performance.

# 6 Discussion Of Results

The evaluation of various recommendation methodologies provided valuable insights into their respective capabilities, limitations, and applicability to our use case. Among all the models tested, the Falcon 3-1B model distinctly outperformed others, demonstrating a superior ability to capture deep semantic relationships and maintain coherence over extended textual contexts. Although quantitative metrics for Falcon 3B-1B were not fully documented in this evaluation, its qualitative performance consistently indicated a more personalized and contextually appropriate recommendation output compared to its counterparts.

BERT was also an excellent choice due to its strength in contextual understanding and semantic representation. However, BERT's architecture, while effective, did not exhibit the same level of user preference alignment or conversational fluency as Falcon 3-1B, particularly in generating recommendations that adapt to nuanced user intents.

In contrast, the Word2Vec model, although computationally efficient and lightweight, revealed significant limitations in recommendation quality. It often mismatched book titles with inappropriate genres, indicating a limited capacity to interpret semantic subtleties and contextual cues. This shortcoming highlights the model's dependency on surface-level co-occurrence rather than deeper linguistic representations.

Traditional NLP techniques such as Bag-of-Words (BoW) and Bag-of-N-Grams further lagged in performance. These models rely on frequency-based representations without considering word order, syntax, or contextual dependencies. Consequently, the recommendations generated through these methods were often irrelevant or misaligned with user interests, confirming their inadequacy for modern, semantically-aware recommendation systems.

In summary, the evaluation underscores a clear hierarchy of effectiveness across the models tested. Falcon 3-1B, with its decoder-only architecture and autoregressive generation, demonstrates the most promising potential for developing intelligent, adaptive, and user-centric book recommendation systems.

# 7 Conclusion, Recommendations and Future Works

## 7.1 Conclusion

After testing a range of recommendation strategies, Falcon3-1B takes the lead as the top performer for our book recommendation system. Its strength lies in its ability to uncover rich semantic relationships and make sense of broader contexts, resulting in suggestions that are both spot-on and personalized. BERT puts up a good fight with its high semantic meaning capturing but it cannot quite keep pace with Falcon's adaptability and user-focused output. Word2Vec, while efficient, falters in maintaining coherence between genres and titles, and traditional methods like BoW and Bag-of-N-Grams fall flat, limited by their reliance on basic word frequencies over deeper understanding. These results underline the value of advanced models like Falcon 3 1B, which blend cutting-edge deep learning with a sharp eye for context and user needs.

## 7.2 Recommendations and Future Work

Proposed Enhancements to the Recommendation System
To strengthen the effectiveness and scalability of the current book recommendation system, several key improvements have been identified. These focus on deepening personalization, enhancing responsiveness, and optimizing system performance for broader deployment.

### 7.2.1 Hybrid Recommendation Approaches

To generate more diverse and relevant suggestions, we recommend integrating hybrid techniques that combine the capabilities of the Falcon 3B language model with collaborative filtering. This approach would allow the system to incorporate both content semantics and user-based behaviour, improving its ability to recommend titles that align with individual preferences.

Additionally, introducing reinforcement learning can help the model adapt dynamically by learning from real-time user interactions such as clicks, ratings, and reading patterns. This continual learning process would enable the system to fine-tune its outputs based on user satisfaction.

Incorporating knowledge graphs is another promising direction, as they can represent relationships between books, authors, genres, and concepts. This structure allows the system to make more informed and context-rich recommendations by understanding how different entities are connected.

### 7.2.2 Real-Time Adaptation and Personalization

To better capture users' current interests, we propose the implementation of real-time personalization mechanisms. By integrating feedback loops that respond to live user behaviour such as browsing activity or recent selections the system can adapt its recommendations instantly to reflect short-term preferences.

Exploring session-based recommendation models can further enhance this responsiveness. Unlike traditional methods that rely on long-term profiles, session-based models focus on understanding and addressing immediate intent, thereby increasing the relevance of recommendations within a single user session.

### 7.2.3 Improving Efficiency and Supporting Edge Deployment

Given the resource-intensive nature of large-scale models like Falcon 3B, it is important to enhance system efficiency. Model distillation offers a practical solution by producing a lighter version of the model that maintains core performance while significantly reducing computational overhead.

Deploying the distilled model on edge devices can also improve user experience by minimizing latency and reducing dependence on cloud infrastructure. This is particularly useful in mobile environments or regions with limited connectivity, enabling the system to deliver quick and consistent recommendations regardless of network conditions.

# 8 References


[1] G. De Gemmis, A. Lops, M. Semeraro, and P. Basile, "Semantics-aware recommendation systems," Proceedings of the European Conference on Artificial Intelligence (ECAI), 2018.

[2] S. Pradeep, S. Jain, and H. Pradhan, "Content-based movie recommendation using metadata," Journal of Artificial Intelligence Research, vol. 36, pp. 217-230, 2019.

[3] Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," Proceedings of the Conference on Neural Information Processing Systems (NeurIPS), 2017.

[4] M. Schick and H. Schütze, "It’s not just size that matters: Pattern-exploiting training for few-shot learning with small LMs," Proceedings of the Conference on Neural Information Processing Systems (NeurIPS), 2021.

[5] F. Naveed, M. K. Abdullah, and Z. Hussain, "A survey on large language models: Architectures, training, and limitations," IEEE Access, vol. 8, pp. 32020-32042, 2020.

[6] Y. Wang, W. Wang, and Z. Li, "A next-generation architecture for LLM-based recommender systems," ACM Computing Surveys, vol. 56, no. 4, pp. 15-28, Apr. 2023.

[7] W. Zhao, X. Yang, L. Wu, and D. Zhang, "How LLMs change recommender system pipelines: A survey," Proceedings of the International Conference on Recommender Systems (RecSys), 2022.

[8] W. Mao, Y. Liu, Z. Zhou, and X. Zhang, "Survey on LLM-based recommender systems: Encoder-based vs. decoder-based architectures," Journal of AI and Data Mining, vol. 5, no. 3, pp. 201-220, 2023.

[9] Y. Wu, Y. Zhang, and L. Lin, "Domain-specific review of large language models for recommendation applications," Journal of E-Commerce and AI, vol. 42, no. 3, pp. 234-247, 2023.

[10] Y. Li, X. Huang, and T. Lu, "Tutorial on applying large language models in recommender systems," Proceedings of the International Conference on Machine Learning (ICML), 2023.

[11] P. Vats, M. Patil, and R. Ahuja, "A review of LLM-based recommendation systems: Performance benefits, challenges, and ethical considerations," Journal of Artificial Intelligence Ethics, vol. 4, pp. 93-105, 2023.

[12] M. Javed, M. A. Malik, and N. G. Y. Lye, "From content-based filtering to LLM-powered recommendations: A review of approaches," Information Retrieval Journal, vol. 40, no. 2, pp. 98-112, Feb. 2023.

[13] X. Ji, X. Zhang, and P. Chen, "GenRec: A generative framework for LLM-based recommendation," Proceedings of the International Conference on Recommender Systems (RecSys), 2023.

[14] S. Friedman, A. Cheng, and L. Liu, "Conversational recommendation systems using large language models," Proceedings of the ACM Conference on Recommender Systems (RecSys), 2022.

[15] W. Dodge, H. Li, and A. L. Goh, "Fine-tuning large language models for recommendation systems: A study on initialization, early stopping, and data ordering," Journal of Machine Learning Research, vol. 24, no. 1, pp. 1217-1239, Jan. 2023.

[16] H. Lv, L. Zhang, and Y. Guo, "Optimizing large language model fine-tuning for recommendation tasks in computationally constrained environments," Journal of AI Research, vol. 26, no. 5, pp. 1224-1239, May 2023.

[17] Y. Lin, T. Zhang, and Z. Chen, "Data-efficient fine-tuning strategies for LLMs in recommendation tasks," IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 1, pp. 56-68, Jan. 2023.

[18] S. M. Acharya, M. Bhat, M. Choudhury, and S. Sharma, "Item-description generation for cold-start recommendations using LLMs," IEEE Transactions on Knowledge and Data Engineering, vol. 34, no. 3, pp. 1234-1247, Mar. 2022.

[19] S. Tekle, N. A. Yimam, and R. R. Salim, "LLM-based song lyric summarization for personalized music recommendations," Proceedings of the International Conference on Recommender Systems, 2023.

[20] Z. Zhang, X. Yao, and L. Wang, "Evaluating the limitations of large language models for recommender systems," IEEE Transactions on Knowledge and Data Engineering, vol. 34, no. 7, pp. 1789-1801, Jul. 2023.

# 9 Appendix

## 9.1 Dataset structure used for training:

Each record looks like this:


```
{
    "instruction": "Recommend books based on user query",
    "input": "I like books similar to 'Atomic Habits' by James Clear in English. Can you recommend more?",
    "output": {
        "book_itself": "Atomic Habits",
        "categories": "Self-help, Productivity",
        "average_rating": 4.8,
        "ratings_count": 150000,
        "language": "English",
        "similar_category_books": ["The Power of Habit", "Deep Work"],
        "same_author_books": ["Millionaire Habits", "Success Habits"],
        "similar_description_books": ["The 7 Habits of Highly Effective People", "Mindset"],
        "high_rated_recommendations": ["Grit", "The Subtle Art of Not Giving a F*ck"]
    }
}
```


*Figure 15: Number of records used for training: 1,844,111 instructions*

## 9.2 Model fine tuning details:

```
# Training Arguments (Optimized)
args = TrainingArguments(
    output_dir='falcon_1B_fine_tuned',
    num_train_epochs=1,
    max_steps=-1,  # Start with 1 epoch to test speed
    per_device_train_batch_size=32,  # Reduced batch size for Falcon 3B
    gradient_accumulation_steps=2,
    warmup_ratio=0.03,
    optim='paged_adamw_8bit',
    logging_steps=20000,
    save_strategy='steps',
    save_steps=50000,
    evaluation_strategy='no',
    learning_rate=5e-5,
    bf16=True,  # Use BF16 for faster computation
    logging_dir='./logs',
    save_total_limit=2,
    load_best_model_at_end=False,
    metric_for_best_model='eval_loss',
    greater_is_better=False,
    eval_accumulation_steps=4
)
```

*Figure 16: Hyperparameters Used for training.*

The fine-tuned model has been pushed to hugging face hub the model is available in below link

URL: https://huggingface.co/sri-lasya/falcon-1B-book-recommendation

## 9.3 Code screenshots

### 9.3.1 Model training code

```python
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, TrainingArguments, Trainer, DataCollatorForLanguageModeling
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from datasets import load_dataset
import torch
import os
import evaluate
import warnings as wr
import numpy as np
from transformers import enable_full_determinism, set_seed
from accelerate import Accelerator, PartialState
import json
import gc

#  Set seeds for reproducibility
seed_value = 42
set_seed(seed_value)
enable_full_determinism(seed_value)
wr.filterwarnings('ignore')

#  Initialize Accelerate
accelerator = Accelerator()
partial_state = PartialState()

#  Load 10% of the dataset for initial runs
train_dataset = load_dataset('json', data_files='train.jsonl', split='train')
eval_dataset = load_dataset('json', data_files='test.jsonl', split='train[:1%]')

#  Falcon 3B Instruct Model and Tokenizer
model_name = 'tiiuae/Falcon3-1B-Instruct'

#  Load model with quantization and device mapping
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map='auto',
    #load_in_8bit=True,
    use_cache=False
)
```

*Figure 17: Model training part 1*

```python
model.config.use_cache = False

print(torch.cuda.memory_summary(device=0, abbreviated=False))

#  Load tokenizer with special tokens
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model.gradient_checkpointing_enable()
#  Reduce sequence length to 512
max_length = 512

#  Prompt formatting function
def formatting_func(example):
    output_str = json.dumps(example['output'], ensure_ascii=False)
    text = (
        f"### Instruction: {example['instruction']}\n"
        f"### Input: {example['input']}\n"
        f"### Output: {output_str}"
    )
    return text

#  Tokenization and prompt generation
def generate_and_tokenize_prompt(prompt):
    result = tokenizer(
        formatting_func(prompt),
        truncation=True,
        max_length=max_length,
        padding="max_length"
    )
    result["labels"] = result["input_ids"].copy()
    return result

#  Tokenize datasets
tokenized_train_dataset = train_dataset.map(generate_and_tokenize_prompt)
tokenized_val_dataset = eval_dataset.map(generate_and_tokenize_prompt)
```

*Figure 18: Model training part 2*

```python
# Check available GPUs and enable parallelism with Accelerate
if partial_state.num_processes > 1:
    model.is_parallelizable = True
    model.model_parallel = True

model.gradient_checkpointing_enable()

# Training Arguments (Optimized)
args = TrainingArguments(
    output_dir='falcon_1B_fine_tuned',
    num_train_epochs=1,
    max_steps=-1,  # Start with 1 epoch to test speed
    per_device_train_batch_size=32,  # Reduced batch size for Falcon 3B
    gradient_accumulation_steps=2,
    warmup_ratio=0.03,
    optim='paged_adamw_8bit',
    logging_steps=20000,
    save_strategy='steps',
    save_steps=50000,
    evaluation_strategy='no',
    learning_rate=5e-5,
    bf16=True,  # Use BF16 for faster computation
    logging_dir='./logs',
    save_total_limit=2,
    load_best_model_at_end=False,
    metric_for_best_model='eval_loss',
    greater_is_better=False,
    eval_accumulation_steps=4
)

#  Load Accuracy Metric
accuracy_metric = evaluate.load("accuracy")

#  Compute Evaluation Metrics
def compute_metrics(eval_preds):
    logits, labels = eval_preds
    predictions = np.argmax(logits, axis=-1).flatten()
```

*Figure 19: Model training part 3*

```
    labels = labels.flatten()

    #  Accuracy Calculation
    accuracy_results = accuracy_metric.compute(predictions=predictions, references=labels)

    #  Perplexity Calculation
    eval_loss = torch.nn.CrossEntropyLoss()(torch.tensor(logits).permute(0, 2, 1), torch.tensor(labels)).item()
    perplexity = np.exp(eval_loss) if eval_loss < 100 else float('inf')

    return {
        "accuracy": accuracy_results['accuracy'],
        "perplexity": perplexity
    }

#  Data Collator for Causal LM
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False
)

#  Initialize Trainer with Accelerate
trainer = Trainer(
    model=model,
    tokenizer=tokenizer,
    args=args,
    train_dataset=tokenized_train_dataset,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

#  Prepare Trainer with Accelerate
trainer = accelerator.prepare(trainer)

#  Start Fine-Tuning
model.config.use_cache = False
trainer.train()
```

*Figure 20: Model training part 4*

```
trainer.train()

#  Save Final Model
trainer.save_model('falcon_1B_fine_tuned_final')

del trainer
gc.collect()
torch.cuda.empty_cache()

#  Push Merged Model to Hugging Face Hub
trainer.push_to_hub('sri-lasya/falcon-1B-book-recommendation')

```

*Figure 21: Model training part 5*

### 9.3.2 Model Evaluation code

```python
from transformers import AutoModelForCausalLM, AutoTokenizer,Trainer, TrainingArguments, DataCollatorForLanguageModeling
import torch
from datasets import load_dataset
from tqdm import tqdm
import numpy as np
import evaluate

model_name = "./falcon_1B_fine_tuned_final"

# Load model with automatic device mapping
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token  # Set pad token

test_dataset = load_dataset('json', data_files='test.jsonl', split='train[:10%]')

# Simplified tokenization (no manual tensor conversion)
def preprocess_function(examples):
    return tokenizer(
        [f"### Instruction: {inst}\n### Input: {inp}\n### Response: {out}"
         for inst, inp, out in zip(examples["instruction"], examples["input"], examples["output"])],
        truncation=True,
        max_length=512,
        padding="max_length",
        return_tensors="pt"
    )

```

*Figure 22:Model evaluation part 1*

```python
tokenized_dataset = test_dataset.map(
    preprocess_function,
    batched=True,
    batch_size=8,  # Increase batch size with automatic device mapping
    remove_columns=test_dataset.column_names
)

# Training arguments with memory optimization
training_args = TrainingArguments(
    output_dir="./eval_results",
    per_device_eval_batch_size=4,
    fp16=torch.cuda.is_available(),
    eval_accumulation_steps=2,
    report_to="none"
)

data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    eval_dataset=tokenized_dataset,
)

results = trainer.evaluate()
print(f"Evaluation Loss: {results['eval_loss']:.4f}")
print(f"Perplexity: {np.exp(results['eval_loss']):.2f}")

rouge = evaluate.load("rouge")
generation_args = {
    "max_new_tokens": 128,
    "do_sample": False,
    "pad_token_id": tokenizer.eos_token_id
}

```

*Figure 23:Model evaluation part 2*

```python
all_predictions = []
all_references = []

for i in tqdm(range(0, len(test_dataset)), desc="Generating Responses"):
    sample = test_dataset[i]
    prompt = f"### Instruction: {sample['instruction']}\n### Input: {sample['input']}\n### Response:"

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.inference_mode():
        outputs = model.generate(
            **inputs,
            **generation_args
        )

    prediction = tokenizer.decode(outputs[0][len(inputs.input_ids[0]):], skip_special_tokens=True)
    all_predictions.append(str(prediction).strip())
    all_references.append(str(sample["output"]).strip())

rouge_scores = rouge.compute(
    predictions=all_predictions,
    references=all_references,
    use_stemmer=True
)

print(f"ROUGE-1: {rouge_scores['rouge1']:.4f}")
print(f"ROUGE-2: {rouge_scores['rouge2']:.4f}")
print(f"ROUGE-L: {rouge_scores['rougeL']:.4f}")
```

*Figure 24: Model evaluation part 3*

## 9.4 Application Screenshots:

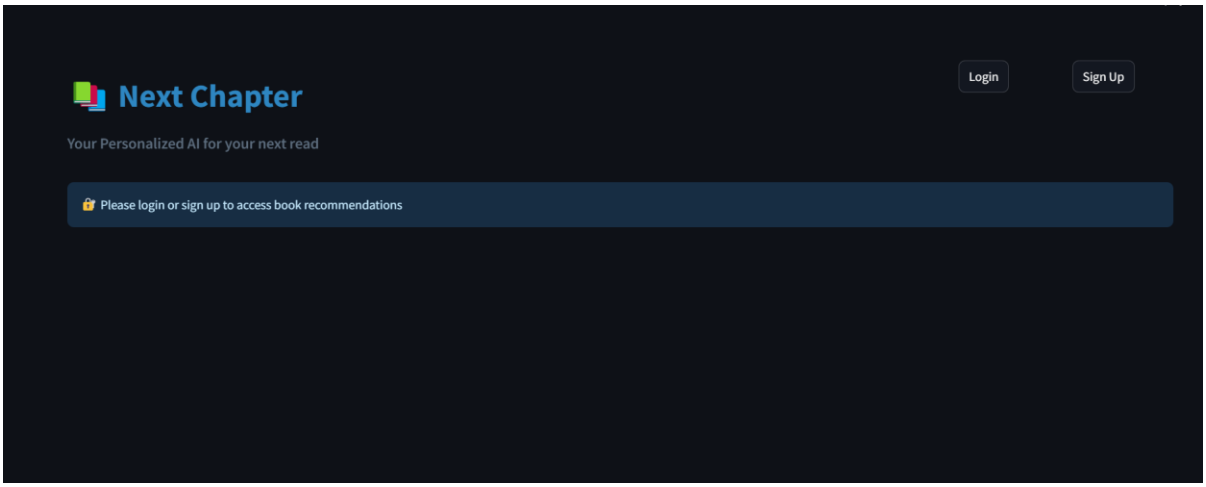



*Figure 25: Main page*

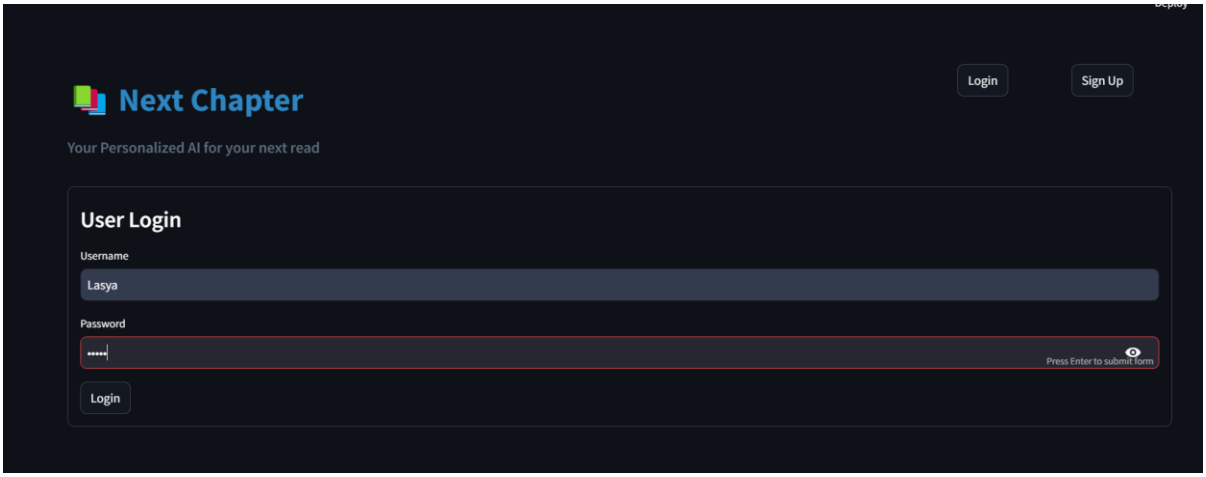



*Figure 26: login page for existing users*

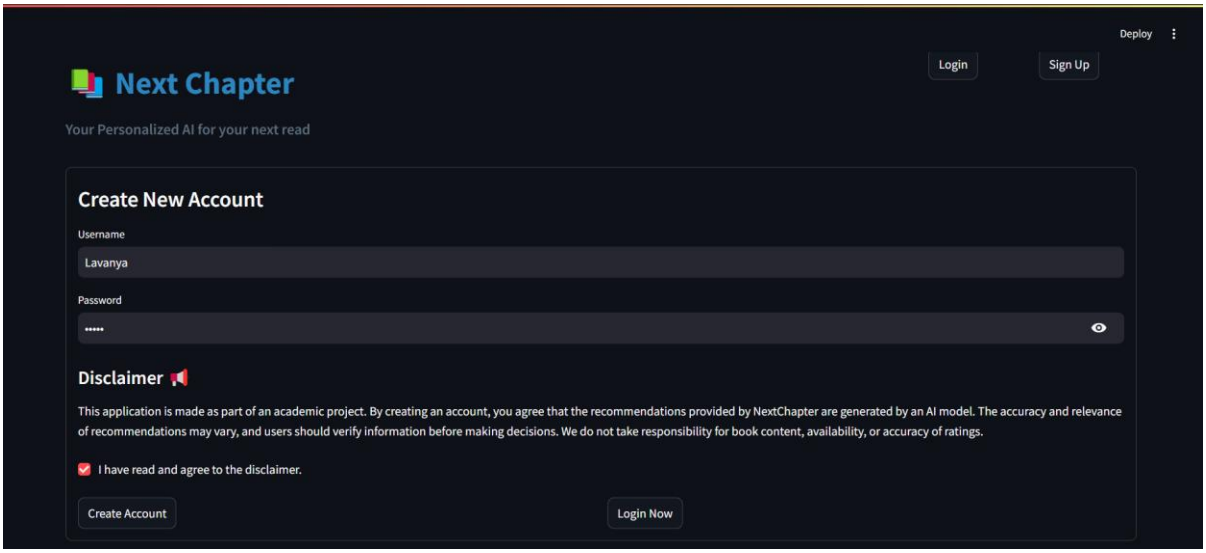



*Figure 27: signup page interface with disclaimer*

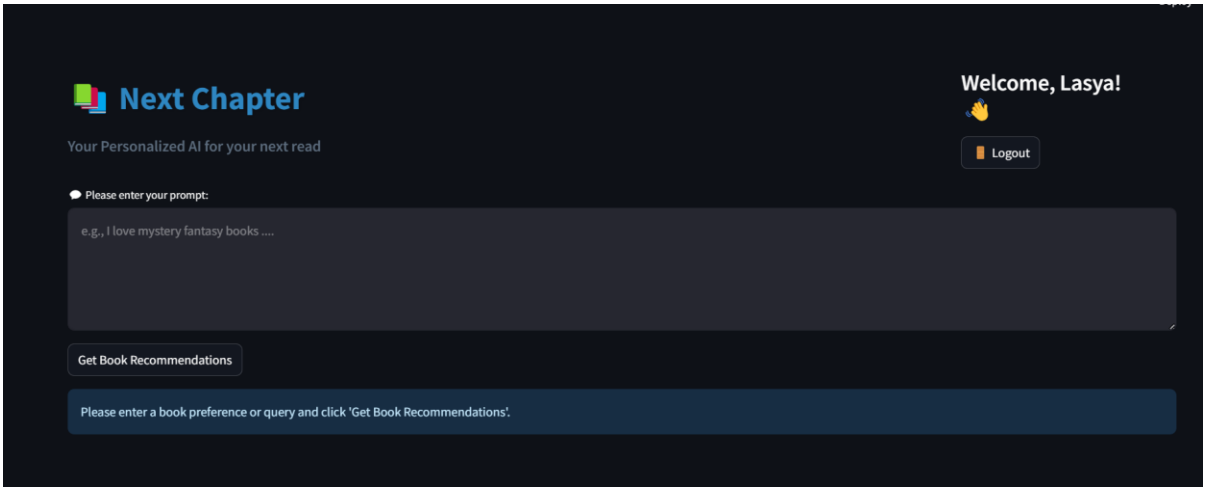



*Figure 28: recommendation page*

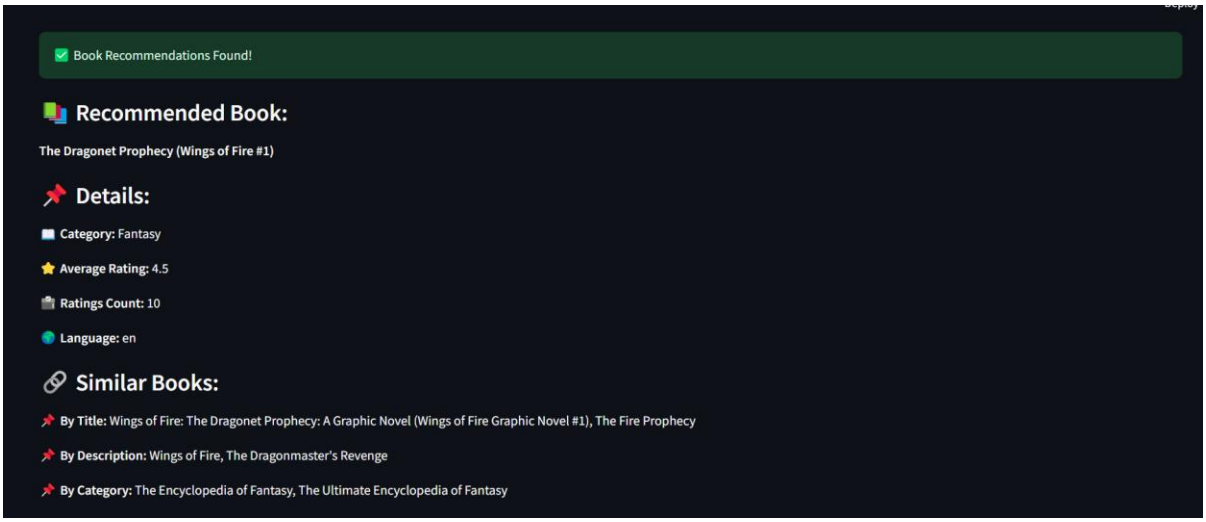



*Figure 29: Output recommendations*

group 6

ORIGINALITY REPORT

| 4% | 1% | 3% | 2% |
|---|---|---|---|
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

PRIMARY SOURCES

| | | |
|---|---|---|
| 1 | Kuldeep Singh Kaswan, Jagjit Singh Dhatterwal, Anand Nayyar. "Digital Personality: A Man Forever - Volume 1: Introduction", CRC Press, 2024<br>Publication | 1% |
| 2 | library.samdu.uz<br>Internet Source | <1% |
| 3 | Sabeen Ahmed, Ian E. Nielsen, Aakash Tripathi, Shamoon Siddiqui, Ravi P. Ramachandran, Ghulam Rasool. "Transformers in Time-Series Analysis: A Tutorial", Circuits, Systems, and Signal Processing, 2023<br>Publication | <1% |
| 4 | Submitted to Capella University<br>Student Paper | <1% |
| 5 | Submitted to University of Northumbria at Newcastle<br>Student Paper | <1% |
| 6 | Konstantina Tzavella, Adrian Diaz, Catharina Olsen, Wim Vranken. "Combining evolution and protein language models for an interpretable cancer driver mutation prediction with D2Deep", Cold Spring Harbor Laboratory, 2024 | <1% |

Publication

| | | |
|---|---|---|
| 7 | Samba Ndiaye. "Chapter 2 NLP and Some Research Results in Senegal", Springer Science and Business Media LLC, 2024<br>Publication | <1 % |
| 8 | Hemant Kumar Soni, Sanjiv Sharma, G. R. Sinha. "Text and Social Media Analytics for Fake News and Hate Speech Detection", CRC Press, 2024<br>Publication | <1 % |
| 9 | Ikram Karabila, Nossayba Darraz, Anas El-Ansari, Nabil Alami, Mostafa El Mallahi. "A hybrid approach combining sentiment analysis and deep learning to mitigate data sparsity in recommender systems", Neurocomputing, 2025<br>Publication | <1 % |
| 10 | Submitted to Colorado State University, Global Campus<br>Student Paper | <1 % |
| 11 | Silin Chen, Tianyang Wang, Xinyuan Song, Bowen Jing, Junjie Yang, Junhao Song, Keyu Chen, Ming Li, Qian Niu, Junyu Liu. "Deep Learning and Machine Learning, Advancing Big Data Analytics and Management: Generative Models", Open Science Framework, 2024<br>Publication | <1 % |
| 12 | Submitted to University of Westminster<br>Student Paper | <1 % |
| 13 | www.pingcap.com<br>Internet Source | |

| | | |
|---|---|---|
| | | <1% |
| 14 | Submitted to Queen Mary and Westfield College<br>Student Paper | <1% |
| 15 | Submitted to British University in Egypt<br>Student Paper | <1% |
| 16 | Submitted to UCL<br>Student Paper | <1% |
| 17 | Submitted to 2U University of Denver MDS<br>Student Paper | <1% |
| 18 | Submitted to Ngee Ann Polytechnic<br>Student Paper | <1% |
| 19 | Submitted to University College Dublin (UCD)<br>Student Paper | <1% |
| 20 | Björn W. Schuller. "Intelligent Audio Analysis", Springer Nature, 2013<br>Publication | <1% |
| 21 | Submitted to CSU, San Jose State University<br>Student Paper | <1% |
| 22 | ikyfxbvl.farmaciasantostefano.it<br>Internet Source | <1% |
| 23 | irjet.net<br>Internet Source | <1% |
| 24 | www.jmir.org<br>Internet Source | <1% |
| 25 | www.mdpi.com<br>Internet Source | <1% |

| | | |
|---|---|---|
| 26 | Babeș-Bolyai University<br>Publication | <1% |
| 27 | Narasimha Rao Vajjhala, Kenneth David Strang. "Cybersecurity in Knowledge Management - Cyberthreats and Solutions", CRC Press, 2025<br>Publication | <1% |
| 28 | theses.hal.science<br>Internet Source | <1% |
| 29 | "Digital Technologies and Applications", Springer Science and Business Media LLC, 2024<br>Publication | <1% |
| 30 | Arcipreste, Beatriz Marques. "Product Complaint Understanding Using NLP Techniques", Universidade do Porto (Portugal), 2024<br>Publication | <1% |
| 31 | Muhammad Usman Hadi, qasem al tashi, Rizwan Qureshi, Abbas Shah et al. "Large Language Models: A Comprehensive Survey of its Applications, Challenges, Limitations, and Future Prospects", Institute of Electrical and Electronics Engineers (IEEE), 2023<br>Publication | <1% |

| | | | |
|---|---|---|---|
| Exclude quotes | On | Exclude matches | < 6 words |
| Exclude bibliography | On | | |

GITAM
DEEMED TO BE UNIVERSITY